

HOSPITAL MANAGEMNET SYSTEM

A DBMS Capstone Project Report submitted in partial fulfillment of the requirements for the award
of the degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Submitted By

K. DEVI SRI CHARAN – 25B11AI570

K. VEERA CHANIKYA – 25B11AI528

D. L. NARASIMHA SAI – 25B11AI247

B. SRIRAM RATNA KUMAR – 25B11AI138

Under the esteemed guidance of

Mr. T.Srinivasulu M.Tech, (Ph.D.)

Assistant Professor



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

ADITYA UNIVERSITY

(Formerly Aditya Engineering College (A))

Aditya Nagar, ADB Road, Surampalem, Andhra Pradesh, India

2026 – 2027

ADITYA UNIVERSITY

(Formerly Aditya Engineering College(A))

Aditya Nagar, ADB Road, Surampalem, Andhra Pradesh, India

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



CERTIFICATE

This is to certify that the DBMS Capstone Project report entitled “Hospital Management System” is being submitted by K. Devi Sri Charan (Roll No. 25B11AI570), K. Veera Chanikya (Roll No.25B11AI528), D. L. Narasimha Sai (Roll No. 25B11AI247) and B. Sriram Ratna Kumar (Roll No. 25B11AI138) in partial fulfillment of the requirements for the award of the B.Tech. degree in Artificial Intelligence and Machine Learning. The work documented in this report was carried out as part of the Capstone Project for the course Database Management Systems (2501IT05) during the academic year 2026- 2027, and records the database design, implementation, and documentation undertaken by the team.

Project Guide

Mr. T. Srinivasulu, M. Tech, (Ph.D.)

Associate professor

Head of the Department

Dr. K. Naga Bhargavi, M. Tech, Ph.D.

Associate Professor&HOD

External Examiner

DECLARATION

We hereby declare that the DBMS Capstone Project entitled “Hospital Management System” is a genuine work carried out by us. The database design, SQL implementation, queries and documentation presented in this report are our own work, completed under the guidance of Mr. T. Srinivasulu, M.Tech(Ph.D) , Department of Artificial Intelligence&Machine Learning, Aditya University, Surampalem. We further declare that this work has not been submitted, in full or in part, for the award of any degree of this or any other educational institution.

Project Associates

K. Devi Sri Charan (25B11AI570)

K. Veera Chanikya (25B11AI528)

D. L. Narasimha Sai (25B11AI247)

B. Sriram Ratna Kumar (25B11AI138)

Date: _____

Place: Surampalem

ACKNOWLEDGEMENT

It is with immense pleasure that we express our indebted gratitude to our project guide Mr. T. Srinivasulu, M.Tech, (Ph.D) , Department of Artificial Intelligence&Machine Learning, who guided us throughout the capstone project. We are thankful for the constant encouragement and valuable suggestions received at every review stage.

We are grateful to K. Naga Bhargavi, M.Tech (Ph.D), Head of the Department, for inspiring us and for arranging the facilities and resources needed for this project.

We extend our sincere thanks to the Management, Dr. M. Sreenivasa Reddy, Deputy Pro-Chancellor, Dr. M. B. Srinivas, Vice-Chancellor, and Dr. A. Ramesh, Pro Vice-Chancellor (Engineering&Sciences), for providing an excellent academic environment.

Not to forget, we thank the Teaching and Non-Teaching Staff, our parents and our friends, who have directly or indirectly supported us in completing this project on time.

Project Associates

K. Devi Sri Charan	25B11AI570
K. Veera Chanikya	25B11AI528
D. L. Narasimha Sai	25B11AI247
B. Sriram Ratna Kumar	25B11AI138

ABSTRACT

This Project belongs to the healthcare domain, where hospitals must manage volumes of patient staff, and financial data every day. In many hospitals, records are still kept on paper or in disconnected spreadsheets, which leads to duplicate entries, lost files, scheduling conflicts, and delays in billing and treatment. The Hospital Management System database solves these problems by storing all information in one structured, consistent, and secure repository. Its purpose is to streamline patient registration, doctor appointments, admissions, prescriptions, and billing, while its scope covers outpatient and inpatient operations within a single hospital. The design contains eight entities, with the major tables being Patient, Doctor, Department, Appointment, Admission, Room, Prescription, and Billing, linked through primary and foreign key constraints. The system is implemented in MySQL, using MySQL Workbench for schema design and ER modelling, and draw.io for diagrams. The main SQL operations include DDL Commands to create tables, DML Commands for inserting, updating, and deleting record, subqueries, aggregate functions, views, and indexes. The outcome is normalized database that reduces redundancy, enforces data integrity, and enables retrieval of patient histories, appointment schedules, and billing reports.

INDEX

Chapter	Title	Page No.
1	Introduction	1-4
	1.1 Introduction to the Domain	1
	1.2 Problem Statement	1-2
	1.3 Motivation and Need for a Database	2
	1.4 Objectives of the Project	3
	1.5 Scope of the Project	3
	1.6 Organization of the Report	4
2	System Study and Tools	5-7
	2.1 Existing System and Its Limitations	5
	2.2 Proposed Database System	6
	2.3 Tools and Technologies Used	7
	2.4 Software and Hardware Requirements	7
3	Database Design (Review - I)	8-32
	3.1 Identified Entities	8
	3.2 Entity Attributes and Attribute Types	9-16
	3.3 ER Diagram	17
	3.4 Relationships and Cardinality	17-18
	3.5 Relational Database Schema	18-30
	3.6 Primary Keys and Foreign Keys	30-31
	3.7 Normalization up to 3NF	31
4	Database Implementation (Review - II)	32-43
	4.1 Database and Table Creation - DDL	32-39
	4.2 Sample Data Insertion - DML	40-41
	4.3 Constraints and Data Integrity	42
	4.4 Data Validation Rules	43
5	SQL Operations and Queries	44-49
	5.1 DQL - Basic Retrieval Queries	44
	5.2 Join Operations	45
	5.3 Aggregate Functions, GROUP BY and HAVING	46

	5.4 Nested and Correlated Subqueries	47
	5.5 Views and Relational Set Operations	48
	5.6 DCL and TCL Operations	49
6	Modules and Functionalities	50-51
7	Results - Sample Outputs and Screenshots	52-54
8	Testing and Data Validation	55
9	GitHub Repository and Team Contribution	56
10	Conclusion and Future Enhancements	57-58
	References	59
	Appendix A - Complete SQL Script	60

LIST OF FIGURES

Figure No.	Title of the Figure	Page No.
Figure 1	Entity Relationship (ER) Diagram of the Hospital Management System	17
Figure 2	Relational Schema (EER) Diagram of hospital_db	18
Figure 3	Database and Tables Created in MySQL Workbench	32-39
Figure 4	Sample Records in the Patients Table	40-41
Figure 5	Join Query: Appointments with Patient, Doctor and Department Details	45
Figure 6	Aggregate Query: Appointments per Department using GROUP BY	46
Figure 7	Subquery: Invoices Above the Average Total Amount	50-51
Figure 8	View Creation: Unpaid and Partially Paid Invoices	52-54

LIST OF TABLES

Table No.	Title of the Table	Page No.
Table 1	Summary of Database Tables and Their Purpose	32
Table 2	Structure of the Admins Table	32-33
Table 3	Structure of the Admissions Table	33-34
Table 4	Structure of the Appointments Table	34
Table 5	Structure of the Departments Table	35
Table 6	Structure of the Doctors Table	35-36
Table 7	Structure of the Inquiries Table	36
Table 8	Structure of the Invoices Table	36-37
Table 9	Structure of the LabTests Table	37-38
Table 10	Structure of the Medicines Table	38-39
Table 11	Structure of the Patients Table	39
Table 12	Structure of the Prescription Details Table	39-40
Table 13	Structure of the Prescriptions Table	40
Table 14	Structure of the Rooms Table	40-41
Table 15	Foreign Key Relationships Between Tables	30-31
Table 16	SQL Operations Implemented	44-49

CHAPTER 1: INTRODUCTION

1.1 Introduction to the Domain

Healthcare is the domain chosen for this project, and the organisation studied is a multi-specialty hospital. A hospital diagnoses, treats and cares for patients through outpatient consultations, inpatient admissions, laboratory diagnostics and pharmacy services. Its work is divided among specialised departments such as Cardiology, Neurology, Orthopedics, Pediatrics, Dermatology, ENT, General Medicine, Gynecology and Obstetrics, Oncology, and Psychiatry. Doctors, nurses, technicians and administrative staff work together across these departments, and each patient's care depends on information moving accurately between them.

Every day the hospital handles a large and varied amount of information. It records patient details such as personal and contact information, blood group, allergies, medical history and emergency contacts, along with professional details for each doctor, including specialisation, qualification, licence number and consultation fee. Appointments are scheduled with a date, time, department and reason for visit, and are later updated with symptoms, diagnosis and status. For inpatients, the hospital tracks the assigned room and bed, the admission and discharge dates, the diagnosis and the treatment summary. Room information, such as room type, bed capacity, occupancy and daily charge, must also be kept current.

The hospital also manages prescriptions, medicine stock, laboratory tests and billing. Prescriptions list the medicines given to a patient with their dosage, frequency and duration, while the pharmacy monitors stock levels, batch numbers and expiry dates. Laboratory tests are ordered, processed and reported with results and verification details. Each visit or stay ends in an invoice that combines consultation, room, lab and medicine charges with tax, discount and payment status. Administrative staff such as receptionists, accountants and managers also handle patient inquiries and complaints. Because this data is sensitive, interconnected and constantly changing, the hospital needs a structured database to keep it accurate, consistent and easy to retrieve.

1.2 Problem Statement

Hospitals handle a constant flow of interconnected information covering patients, doctors, appointments, admissions, rooms, prescriptions, medicines, laboratory tests

and billing. In many hospitals this data is still kept in paper registers and separate departmental spreadsheets, with no single source of truth. As a result, the same patient details are entered repeatedly, records drift out of step when only one copy is corrected, and even simple questions become slow to answer. Examples are which patients have unpaid invoices, which rooms are free, or which medicines are running low. Nothing checks the data at entry either, so duplicate records, invalid values and entries that refer to non-existent patients or doctors can go unnoticed. These problems cause scheduling errors, delayed billing, wasted staff time and, more seriously, incomplete medical information at the point of care.

This project addresses the problem by designing and implementing a relational Hospital Management System database in MySQL. It must store all core hospital data in 13 related tables and remove redundancy through normalisation. It must enforce accuracy through primary keys, foreign keys, UNIQUE and CHECK constraints, and allow quick retrieval of patient histories, appointment schedules, room occupancy, medicine stock and billing summaries using SQL queries, joins, aggregate functions, subqueries and views. The aim is a consistent, reliable and easily searchable database that supports the daily work of both clinical and administrative staff.

1.3 Motivation and Need for a Database

A hospital's data is not a set of independent lists but a web of relationships. One patient can have many appointments, admissions, prescriptions, lab tests and invoices, and each admission links a patient to a doctor, a room and the staff member who created it. Files and spreadsheets store each list separately, so these links can only be kept by copying values by hand, and nothing stops them from breaking. A DBMS models the relationships directly through primary and foreign keys, so a bill cannot exist without a valid patient and a prescription cannot name a medicine that is not in stock records. Spreadsheets also cannot answer questions that cut across several lists. A question such as which departments earned the most from unpaid or partially paid invoices needs joins and grouping, which SQL performs in a single query but which would take a spreadsheet user repeated manual lookups.

A hospital is also a busy, multi-user environment where many staff work on the same data at once. Receptionists book appointments, wards update admissions, the laboratory uploads results and accountants generate invoices at the same time. With shared files, two people editing the same record can overwrite each other's changes

or leave the file corrupted. A DBMS manages concurrent access, and its transaction support ensures that a multi-step operation, such as creating an invoice and recording its payment, is either fully completed or fully undone, never left half-finished. It also provides automatic backup and recovery, so a system failure does not mean the loss of medical and financial records.

Finally, hospital data is sensitive and legally important, so it needs controlled access and long-term reliability. Anyone who can open a spreadsheet can usually see and change everything in it, whereas a DBMS allows access to be restricted by user and role, for example letting a receptionist manage appointments but not view billing details, or an accountant manage invoices but not medical history. A DBMS also scales far better: as records grow from hundreds to hundreds of thousands, indexes keep searches fast, whereas spreadsheets slow down and become unreliable. It also separates the data structure from the applications using it, so new tables or fields can be added later without rewriting everything. For these reasons a DBMS such as MySQL is the appropriate foundation for a hospital, and files or spreadsheets are not.

1.4 Objectives of the Project

- 1. To design a normalized relational database for the selected domain with at least six entities.**
- 2. To implement the database with appropriate primary keys, foreign keys and integrity constraints.**
- 3. To populate the database with realistic sample data of over 1,000 records across all 13 tables, including 100 patients, 30 doctors and 180 appointments, so that every query can be tested on meaningful data.**
- 4. To implement and execute at least 10 SQL queries covering DDL and DML commands, SELECT with WHERE, ORDER BY and GROUP BY, aggregate functions, multi-table joins, subqueries, views and indexes.**
- 5. To generate at least three operational reports (appointment schedules by department, room occupancy, and unpaid or partially paid invoices) and verify**

that the database rejects invalid entries, such as a duplicate doctor licence number or an appointment for a non-existent patient.

1.5 Scope of the Project

The database covers the core clinical and administrative operations of a single multi-specialty hospital. It stores patient registration data, doctors and departments, appointments from booking through diagnosis and final status, and inpatient admissions with room and bed allocation, dates and treatment summaries. It also holds room details, prescriptions with their medicine entries, medicine stock with batch and expiry information, laboratory tests with results, and invoices that combine consultation, room, lab and medicine charges with tax, discount and payment status. Administrative accounts, roles, and public inquiries are recorded as well. These 13 tables are linked by primary and foreign keys, protected by integrity constraints, and support queries, joins, aggregate functions, subqueries and views.

The database does not cover everything a hospital needs. It has no records for nurses or other clinical staff, and no shift scheduling, attendance, payroll, equipment, ambulance, blood bank or operating theatre management beyond room allocation. Insurance is stored only as a payment mode, without insurer, policy or claim details. Medicine suppliers and department heads are kept as plain ID numbers with no supplier table or enforced link, and lab reports are stored as file paths rather than as files. The project delivers only the database layer, with no user interface, access-control system or integration with external health systems, and it is limited to one hospital using sample data.

1.6 Organization of the Report

This report is organized into ten chapters. Chapter 2 presents the system study and the tools used. Chapter 3 covers the complete database design including the ER diagram, relational schema and normalization. Chapter 4 describes the database implementation with DDL and sample data. Chapter 5 presents the SQL operations and queries executed. Chapter 6 explains the modules and functionalities. Chapter 7 presents the results with sample outputs. Chapter 8 covers testing and data

validation. Chapter 9 details the GitHub repository and the contribution of each team member. Chapter 10 concludes the report and outlines future enhancements.

CHAPTER 2: SYSTEM STUDY AND TOOLS

2.1 Existing System and Its Limitations

Existing system (without a database)

In a hospital without a proper database, information is kept in paper registers, patient files and separate spreadsheets maintained by each section. The reception desk writes registrations and appointments in a register. The wards keep admission notes in patient files, and the laboratory records test orders and results on its own sheets. The pharmacy tracks medicine stock in a stock book, and the accounts section prepares invoices by hand or in a standalone spreadsheet. Each section keeps its own copy of the details it needs and shares information with the others by passing papers, copying values or making phone calls. A patient's complete history therefore exists only by collecting pieces from several places, and staff have to search, cross-check and combine them manually for every task, from checking a bed's availability to preparing a bill.

Limitations of the existing system

- **Data redundancy:** The same patient details are written repeatedly in the appointment register, admission file, lab sheet and invoice, wasting time and storage.
- **Data inconsistency:** Copies are updated separately, so a corrected phone number, diagnosis or payment status may change in one place and stay outdated in another.
- **Difficult retrieval:** Simple questions, such as which patients have unpaid invoices, which rooms are free, or which medicines are low in stock, mean searching through files or combining spreadsheets by hand.
- **No integrity checks:** Nothing stops duplicate doctor licence numbers, invalid values, or an appointment for a patient who was never registered.
- **Weak relationships between data:** Links between patients, doctors, rooms, prescriptions and bills exist only in staff memory or handwritten references, so they easily break.
- **No safe multi-user access:** Two staff editing the same shared file can overwrite each other's changes, and paper records can be used by only one person at a time.

- **Poor security and privacy:** Paper files are easy to misplace or read without permission, and spreadsheets give anyone with access the ability to see and change everything.
- **Risk of data loss:** Files can be lost, damaged or corrupted, and there is no automatic backup or recovery for medical and financial records.
- **Slow reporting and billing errors:** Totals, taxes and discounts calculated by hand cause mistakes and delays in invoices and management reports.
- **Poor scalability:** As patients and records grow, searching and maintaining files becomes slower and less reliable.

2.2 Proposed Database System

Proposed system

The proposed system is a relational Hospital Management System database built in MySQL (hospital_db). It replaces scattered registers and spreadsheets with one central database of 13 related tables: Patients, Doctors, Departments, Appointments, Admissions, Rooms, Prescriptions, PrescriptionDetails, Medicines, LabTests, Invoices, Admins and Inquiries. Each fact is stored once and referenced through keys. For example, a patient's details live only in the Patients table, while appointments, admissions, prescriptions, lab tests and invoices link back to that record through the patient ID.

The database enforces its own rules through primary keys, foreign keys, and UNIQUE, CHECK, NOT NULL and DEFAULT constraints. Staff retrieve and analyse information with SQL, using queries, joins, aggregate functions, subqueries, views and indexes. Reception, wards, laboratory, pharmacy and accounts all work from the same data, so every section sees the same up-to-date record.

Advantages over the existing system

- **Reduced redundancy:** Details are entered once and referenced everywhere, instead of being rewritten in every register and sheet.
- **Consistent data:** One update is reflected in every related record, so the copies can no longer drift apart.

- **Fast retrieval:** Questions such as which patients have unpaid invoices, which rooms are free, or which medicines are low in stock are answered by a single SQL query instead of a manual search.
- **Built-in integrity:** Foreign keys stop records from referring to a patient, doctor or room that does not exist. UNIQUE and CHECK constraints reject duplicate licence numbers and invalid statuses.
- **Connected data:** Relationships between patients, doctors, rooms, prescriptions and bills are stored in the database, so they cannot be forgotten or broken by hand.
- **Safe multi-user access:** Many staff can work on the data at the same time without overwriting each other, and transactions keep multi-step operations complete or fully undone.
- **Better security and recovery:** MySQL can restrict access by user, and standard backup and recovery protect medical and financial records from loss.
- **Accurate reporting and billing:** Invoice totals, dues and payment status are stored and can be summarised by SQL, which reduces calculation mistakes and delays.
- **Scalability and flexibility:** Indexes keep searches fast as records grow, and new tables or fields can be added later without redesigning the whole system.

2.3 Tools and Technologies Used

Table 2.1: Tools and Technologies Used

Category	Tool / Technology Used	Version / Remarks
Database Server (DBMS)	MySQL with InnoDB engine	8.0.x; database hospital_db, utf8mb4 character set
Query / Admin Tool	MySQL Workbench	Used to run the SQL script and view query output
ER Diagram Tool	draw.io (diagrams.net)	Used for the ER diagram and relational

		schema
Front-end	Not used	Database-only project; queries run directly in the SQL client
Version Control	Git and GitHub	Stores the SQL script and project documentation
Documentation	Microsoft Word	Used to prepare the final report

2.4 Software and Hardware Requirements

Table 2.2: Software and Hardware Requirements

Requirement Type	Specification
Operating System	Windows 11 (64-bit), 22H2 or later; Windows 10 (64-bit) also supported
Database Server	MySQL Community Server 8.0.16 or later; InnoDB storage engine; utf8mb4 character set; default port 3306
Query / Admin Tool	MySQL Workbench 8.0.x (requires Microsoft Visual C++ 2019 Redistributable)
ER Diagram Tool	draw.io (diagrams.net), web or desktop version
Version Control	Git 2.x and a GitHub account
Documentation	Microsoft Word 2019 or Microsoft 365
Web Browser	Google Chrome or Microsoft Edge (latest version)
Processor	Intel Core i3 (or AMD Ryzen 3), 2.0 GHz, dual-core
RAM	4GB
Storage	2 GB free disk space (HDD)
Display	1366 x 768 resolution
Input Devices	Keyboard and mouse
Network	Internet connection for GitHub and draw.io

CHAPTER 3: DATABASE DESIGN

(Deliverables of Review – I)

3.1 Identified Entities

Minimum six entities are required. Give a one-line description of each.

Table 3.1: Entities Identified for the Selected Domain

S.No	Entity Name	Description
1.	Patients	Stores personal, contact, blood group, allergy and medical history details of registered patients.
2.	Doctors	Stores each doctor's specialization, qualification, license number and consultation fee, linked to a department.
3.	Departments	Stores the hospital's departments with their floor, phone extension and head doctor.
4.	Appointments	Records patient-doctor consultations with date, time, type, status and diagnosis.
5.	Admissions	Records inpatient stays with the assigned doctor, room, bed, dates, diagnosis and treatment summary.
6.	Rooms	Stores room type, floor, bed capacity, occupancy, daily charge and availability status.
7.	Prescriptions	Stores prescriptions issued by doctors to patients, linked to an appointment or an admission.
8.	PrescriptionDetails	Lists the individual medicines in a prescription with dosage, frequency, duration and instructions.
9.	Medicines	Stores the medicine inventory with batch number, stock quantity, unit price and expiry date.
10.	LabTests	Records tests ordered for patients with results, report status, cost and the technician handling them.
11.	Invoices	Stores bills combining consultation, room, lab and medicine charges with tax, discount and payment status.
12.	Admins	Stores administrative staff accounts with their roles, such as Receptionist, Accountant, HR and Manager.
13.	Inquiries	Records inquiries and complaints from the public with type, priority, assigned staff and resolution status.

3.2 Entity Attributes and Attribute Types

List every attribute of every entity. Mark the key column as PK, FK or —, and classify each attribute as Simple,

Table 3.2: Attributes of Each Entity

Entity	Attribute	Data Type	Key (PK / FK / —)	Attribute Type
Patients	patient_id	INT	PK	Simple
	first_name	VARCHAR(255)	—	Simple
	last_name	VARCHAR(255)	—	Simple
	gender	VARCHAR(255)	—	Simple
	dob	VARCHAR(255)	—	Simple
	blood_group	VARCHAR(255)	—	Simple
	phone	VARCHAR(255)	—	Simple
	email	VARCHAR(255)	—	Simple
	address	VARCHAR(255)	—	Composite
	city	VARCHAR(255)	—	Simple
	state	VARCHAR(255)	—	Simple
	pincode	VARCHAR(255)	—	Simple
	emergency_ contact_name	VARCHAR(255)	—	Simple
	emergency_ contact_phone	VARCHAR(255)	—	Simple
	allergies	VARCHAR(255)	—	Multi-valued
	medical_history	VARCHAR(255)	—	Multi-valued
	registration_date	VARCHAR(255)	—	Simple
	aadhar_or_ id_number	VARCHAR(255)	—	Simple
	status	VARCHAR(255)	—	Simple
	created_at	DATETIME	—	Simple

Entity	Attribute	Data Type	Key (PK / FK / —)	Attribute Type
Doctors	doctor_id	INT	PK	Simple
	first_name	VARCHAR(255)	—	Simple
	last_name	VARCHAR(255)	—	Simple
	gender	VARCHAR(255)	—	Simple
	dob	VARCHAR(255)	—	Simple
	phone	VARCHAR(255)	—	Simple
	email	VARCHAR(255)	—	Simple
	address	VARCHAR(255)	—	Composite
	specialization	VARCHAR(255)	—	Simple
	qualification	VARCHAR(255)	—	Multi-valued
	department_id	INT	FK	Simple
	experience_years	INT	—	Simple
	consultation_fee	DECIMAL(10,2)	—	Simple
	joining_date	VARCHAR(255)	—	Simple
	status	VARCHAR(255)	—	Simple
	license_number	VARCHAR(255)	—	Simple
	created_at	DATETIME	—	Simple
	updated_at	DATETIME	—	Simple

Entity	Attribute	Data Type	Key (PK / FK / —)	Attribute Type
Departments	department_id	INT	PK	Simple
	department_name	VARCHAR(255)	—	Simple
	description	VARCHAR(255)	—	Simple
	head_doctor_id	INT	—	Simple
	floor_number	INT	—	Simple
	phone_extension	VARCHAR(255)	—	Simple
	status	VARCHAR(255)	—	Simple
	created_at	DATETIME	—	Simple

Entity	Attribute	Data Type	Key (PK / FK / —)	Attribute Type
Appointments	appointment_id	INT	PK	Simple
	patient_id	INT	FK	Simple
	doctor_id	INT	FK	Simple
	department_id	INT	FK	Simple
	appointment_date	VARCHAR(255)	—	Simple
	appointment_time	VARCHAR(255)	—	Simple
	appointment_type	VARCHAR(255)	—	Simple
	status	VARCHAR(255)	—	Simple
	reason_for_visit	VARCHAR(255)	—	Simple
	symptoms	VARCHAR(255)	—	Simple
	diagnosis	VARCHAR(255)	—	Simple
	notes	VARCHAR(255)	—	Simple
	created_by	INT	FK	Simple
	created_at	DATETIME	—	Simple
	updated_at	DATETIME	—	Simple

Entity	Attribute	Data Type	Key (PK / FK / —)	Attribute Type
Rooms	room_id	INT	PK	Simple
	room_number	VARCHAR(255)	—	Simple
	room_type	VARCHAR(255)	—	Simple
	floor_number	INT	—	Simple
	department_id	INT	FK	Simple
	bed_capacity	INT	—	Simple
	beds_occupied	INT	—	Simple
	daily_charge	DECIMAL(10,2)	—	Simple
	status	VARCHAR(255)	—	Simple
	created_at	DATETIME	—	Simple

Entity	Attribute	Data Type	Key (PK / FK / —)	Attribute Type
Admissions	admission_id	INT	PK	Simple
	patient_id	INT	FK	Simple
	doctor_id	INT	FK	Simple
	room_id	INT	FK	Simple
	bed_number	VARCHAR(255)	—	Simple
	admission_date	VARCHAR(255)	—	Simple
	discharge_date	VARCHAR(255)	—	Simple
	reason_for_admission	VARCHAR(255)	—	Simple
	diagnosis	VARCHAR(255)	—	Simple
	treatment_summary	VARCHAR(255)	—	Simple
	admission_type	VARCHAR(255)	—	Simple
	status	VARCHAR(255)	—	Simple
	attendant_name	VARCHAR(255)	—	Simple
	attendant_phone	VARCHAR(255)	—	Simple
	created_by	INT	FK	Simple
	created_at	DATETIME	—	Simple

Entity	Attribute	Data Type	Key (PK / FK / —)	Attribute Type
Prescriptions	prescription_id	INT	PK	Simple
	patient_id	INT	FK	Simple
	doctor_id	INT	FK	Simple
	appointment_id	INT	FK	Simple
	admission_id	INT	FK	Simple
	prescription_date	VARCHAR(255)	—	Simple
	diagnosis	VARCHAR(255)	—	Simple
	notes	VARCHAR(255)	—	Simple
	status	VARCHAR(255)	—	Simple
	created_at	DATETIME	—	Simple

Entity	Attribute	Data Type	Key (PK / FK / —)	Attribute Type
Prescription Details	prescription_detail_id	INT	PK	Simple
	prescription_id	INT	FK	Simple
	medicine_id	INT	FK	Simple
	dosage	VARCHAR(255)	—	Simple
	frequency	VARCHAR(255)	—	Simple
	duration	VARCHAR(255)	—	Simple
	instructions	VARCHAR(255)	—	Simple

Entity	Attribute	Data Type	Key (PK / FK / —)	Attribute Type
Medicines	medicine_id	INT	PK	Simple
	medicine_name	VARCHAR(255)	—	Simple
	generic_name	VARCHAR(255)	—	Simple
	category	VARCHAR(255)	—	Simple
	manufacturer	VARCHAR(255)	—	Simple
	batch_number	VARCHAR(255)	—	Simple
	unit	VARCHAR(255)	—	Simple
	stock_quantity	INT	—	Simple
	reorder_level	INT	—	Simple
	unit_price	DECIMAL(10,2)	—	Simple
	expiry_date	VARCHAR(255)	—	Simple
	manufacture_date	VARCHAR(255)	—	Simple
	supplier_id	INT	—	Simple
	status	VARCHAR(255)	—	Simple
	created_at	DATETIME	—	Simple

Entity	Attribute	Data Type	Key (PK / FK / —)	Attribute Type
LabTests	lab_test_id	INT	PK	Simple
	patient_id	INT	FK	Simple
	doctor_id	INT	FK	Simple
	test_name	VARCHAR(255)	—	Simple
	test_category	VARCHAR(255)	—	Simple
	test_date	VARCHAR(255)	—	Simple
	sample_collected_date	VARCHAR(255)	—	Simple
	result	VARCHAR(255)	—	Simple
	normal_range	VARCHAR(255)	—	Simple
	report	VARCHAR(255)	—	Simple
	report_generated_date	VARCHAR(255)	—	Simple
	report_format	VARCHAR(255)	—	Simple
	report_verified_by	INT	FK	Simple
	is_report_ready	INT	—	Simple
	status	VARCHAR(255)	—	Simple
	test_cost	DECIMAL(10,2)	—	Simple
	technician_id	INT	FK	Simple
	remarks	VARCHAR(255)	—	Simple
	created_at	DATETIME	—	Simple

Entity	Attribute	Data Type	Key (PK / FK / —)	Attribute Type
Admins	admin_id	INT	PK	Simple
	first_name	VARCHAR(255)	—	Simple
	last_name	VARCHAR(255)	—	Simple
	username	VARCHAR(255)	—	Simple
	password_hash	VARCHAR(255)	—	Simple
	email	VARCHAR(255)	—	Simple
	phone	VARCHAR(255)	—	Simple
	role	VARCHAR(255)	—	Simple
	department_id	INT	FK	Simple
	status	VARCHAR(255)	—	Simple
	last_login	VARCHAR(255)	—	Simple
	created_at	DATETIME	—	Simple

Entity	Attribute	Data Type	Key (PK / FK / —)	Attribute Type
Inquiries	inquiry_id	INT	PK	Simple
	name	VARCHAR(255)	—	Composite
	phone	VARCHAR(255)	—	Simple
	email	VARCHAR(255)	—	Simple
	inquiry_type	VARCHAR(255)	—	Simple
	subject	VARCHAR(255)	—	Simple
	message	VARCHAR(255)	—	Simple
	department_id	INT	FK	Simple
	assigned_to	INT	FK	Simple
	status	VARCHAR(255)	—	Simple
	priority	VARCHAR(255)	—	Simple
	response	VARCHAR(255)	—	Simple
	inquiry_date	VARCHAR(255)	—	Simple
	resolved_date	VARCHAR(255)	—	Simple

Entity	Attribute	Data Type	Key (PK / FK / —)	Attribute Type
Invoices	invoice_id	INT	PK	Simple
	patient_id	INT	FK	Simple
	appointment_id	INT	FK	Simple
	admission_id	INT	FK	Simple
	invoice_date	VARCHAR(255)	—	Simple
	consultation_charges	DECIMAL(10,2)	—	Simple
	room_charges	DECIMAL(10,2)	—	Simple
	lab_test_charges	DECIMAL(10,2)	—	Simple
	medicine_charges	DECIMAL(10,2)	—	Simple
	other_charges	DECIMAL(10,2)	—	Simple
	discount	DECIMAL(10,2)	—	Simple
	tax_amount	DECIMAL(10,2)	—	Derived
	total_amount	DECIMAL(10,2)	—	Derived
	paid_amount	DECIMAL(10,2)	—	Simple
	due_amount	DECIMAL(10,2)	—	Derived
	payment_mode	VARCHAR(255)	—	Simple
	payment_status	VARCHAR(255)	—	Simple
	generated_by	INT	FK	Simple
	created_at	DATETIME	—	Simple

3.3 ER Diagram

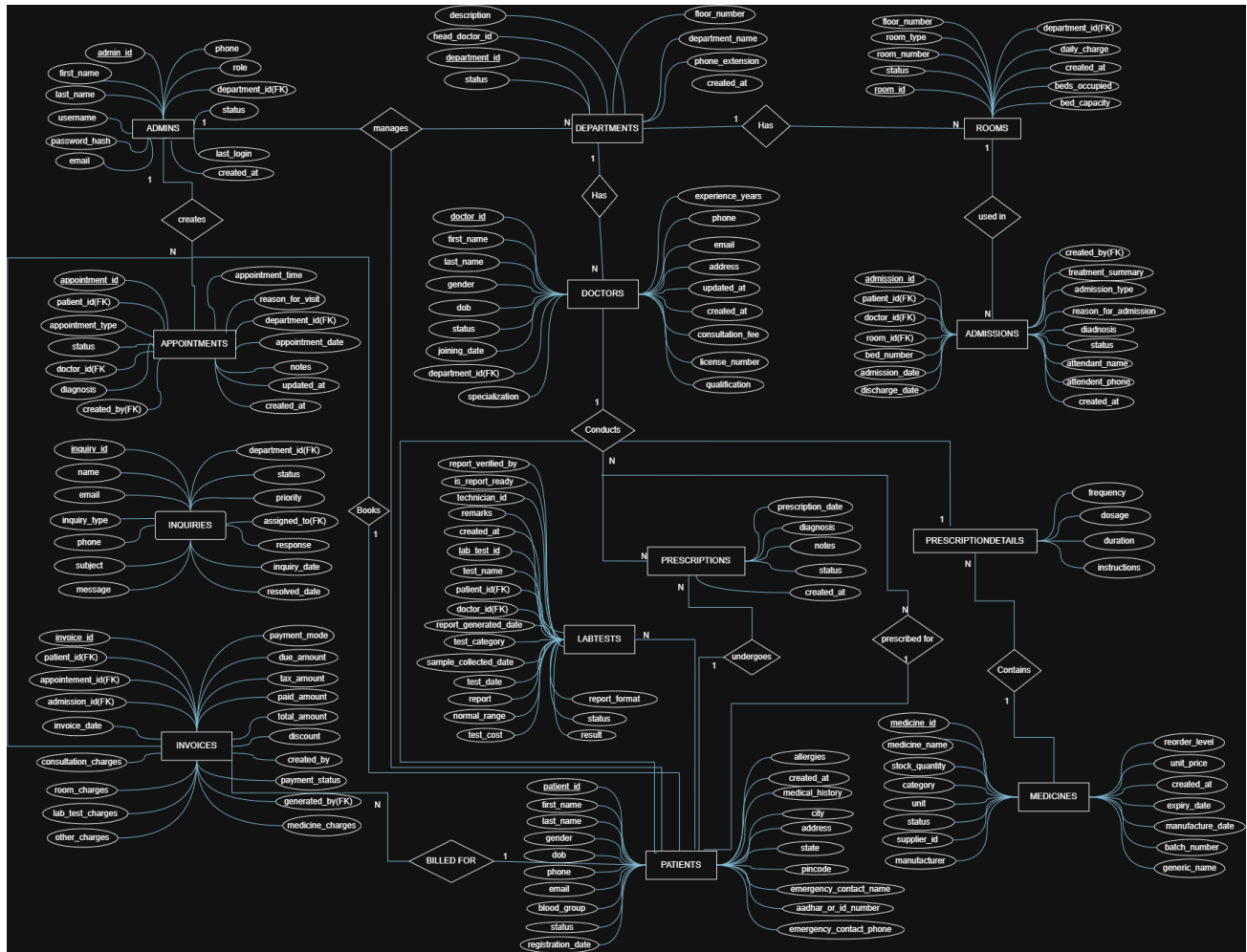


Fig. 3.1: ER Diagram of the System

3.4 Relationships and Cardinality

Table 3.3: Relationships and Cardinality

Entity 1	Relationship	Entity 2	Cardinality (1:1 / 1:N / M:N)	Participation
Department	Has	Doctor	1:N	Partial – Partial
Doctor	Belongs to	Department	N:1	Partial – Partial
Patient	Books	Appointment	1:N	Partial – Partial
Appointment	Is Booked by	Patient	N:1	Partial – Partial

Patient	Has	Admission	1:N	Partial – Partial
Admission	Belongs to	Patient	N:1	Partial – Partial
Patient	Undergoes	Lab Test	1:N	Partial – Partial
Lab Test	Is performed for	Patient	N:1	Partial – Partial
Prescription	Contains	Prescription Details	1:N	Partial – Partial
Prescription Details	Refers to	Medicine	N:1	Partial – Partial

3.5 Relational Database Schema

Table 3.4: Relational Database Schema

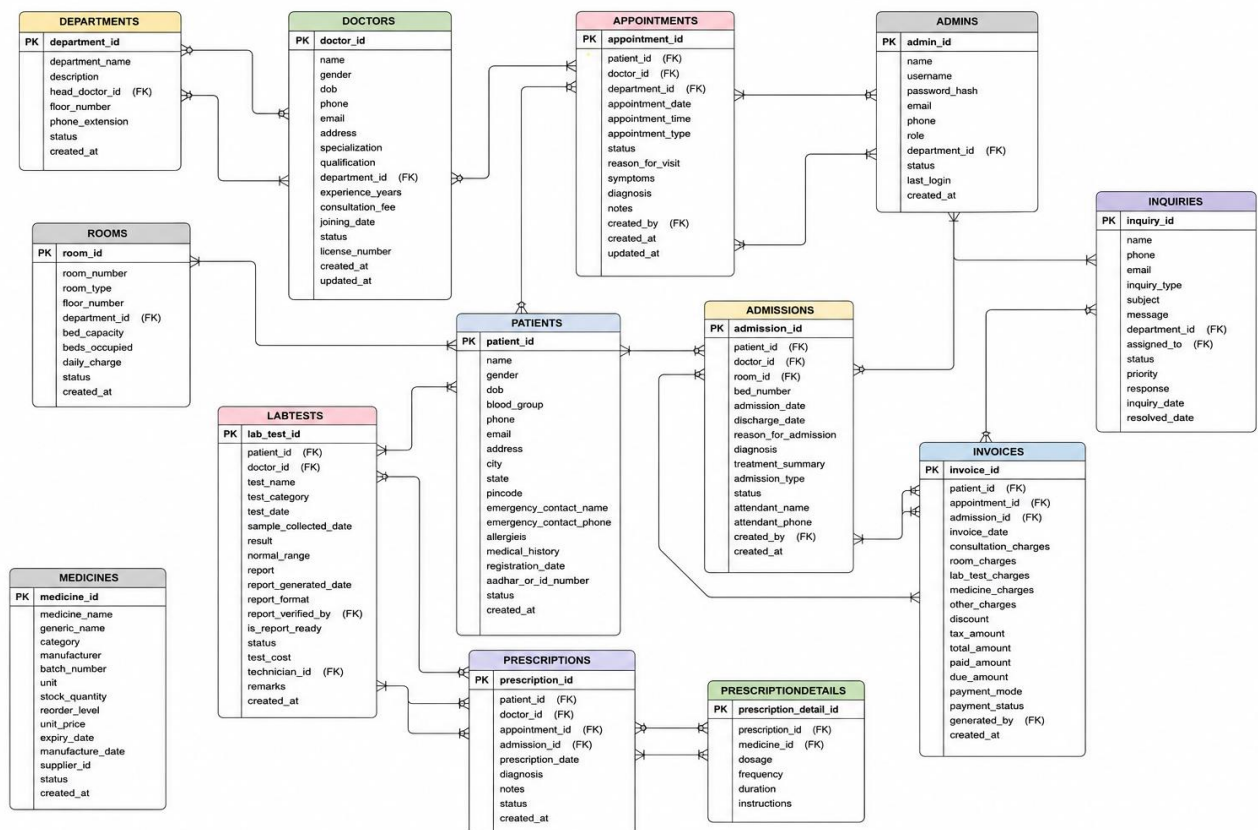


Table Name	Column Name	Data Type	Key (PK / FK / —)	Constraints
Patients	patient_id	INT	PK	PRIMARY KEY, AUTO_INCREMENT
	first_name	VARCHAR(255)	—	NOT NULL
	last_name	VARCHAR(255)	—	NOT NULL
	gender	VARCHAR(255)	—	CHECK IN ('M','F','Other')
	dob	VARCHAR(255)	—	None
	blood_group	VARCHAR(255)	—	None
	phone	VARCHAR(255)	—	None
	email	VARCHAR(255)	—	None
	address	VARCHAR(255)	—	None
	city	VARCHAR(255)	—	None
	state	VARCHAR(255)	—	None
	pincode	VARCHAR(255)	—	None
	emergency_contact_name	VARCHAR(255)	—	None
	emergency_contact_phone	VARCHAR(255)	—	None
	allergies	VARCHAR(255)	—	None
	medical_history	VARCHAR(255)	—	None
	registration_date	VARCHAR(255)	—	None
	aadhar_or_id_number	VARCHAR(255)	—	None
	status	VARCHAR(255)	—	DEFAULT 'Active' CHECK IN ('Active','Inactive','Deceased')
	created_at	DATETIME	—	DEFAULT CURRENT_TIMESTAMP

Table Name	Column Name	Data Type	Key (PK / FK / —)	Constraints
Doctors	doctor_id	INT	PK	PRIMARY KEY, AUTO_INCREMENT
	first_name	VARCHAR(255)	—	NOT NULL
	last_name	VARCHAR(255)	—	NOT NULL
	gender	VARCHAR(255)	—	CHECK IN ('M','F','Other')
	dob	VARCHAR(255)	—	None
	phone	VARCHAR(255)	—	None
	email	VARCHAR(255)	—	UNIQUE
	address	VARCHAR(255)	—	None
	specialization	VARCHAR(255)	—	None
	qualification	VARCHAR(255)	—	None
	department_id	INT	FK	REFERENCES Departments(department_id)
	experience_years	INT	—	None
	consultation_fee	DECIMAL(10,2)	—	None
	joining_date	VARCHAR(255)	—	None
	status	VARCHAR(255)	—	DEFAULT 'Active' CHECK IN ('Active','Inactive','On Leave')
	license_number	VARCHAR(255)	—	UNIQUE
	created_at	DATETIME	—	DEFAULT CURRENT_TIMESTAMP
	updated_at	DATETIME	—	DEFAULT CURRENT_TIMESTAMP

Table Name	Column Name	Data Type	Key (PK / FK / —)	Constraints
Departments	department_id	INT	PK	PRIMARY KEY, AUTO_INCREMENT
	department_name	VARCHAR(255)	—	NOT NULL
	description	VARCHAR(255)	—	None
	head_doctor_id	INT	—	None
	floor_number	INT	—	None
	phone_extension	VARCHAR(255)	—	None
	status	VARCHAR(255)	—	DEFAULT 'Active' CHECK IN ('Active','Inactive')
	created_at	DATETIME	—	DEFAULT CURRENT_TIMESTAMP

Table Name	Column Name	Data Type	Key (PK / FK / —)	Constraints
Appointments	appointment_id	INT	PK	PRIMARY KEY, AUTO_INCREMENT
	patient_id	INT	FK	REFERENCES Patients(patient_id)
	doctor_id	INT	FK	REFERENCES Doctors(doctor_id)
	department_id	INT	FK	REFERENCES Departments(department_id)
	appointment_date	VARCHAR(255)	—	None
	appointment_time	VARCHAR(255)	—	None
	appointment_type	VARCHAR(255)	—	CHECK IN ('New','Follow-up')
	status	VARCHAR(255)	—	CHECK IN ('Scheduled', 'Completed', 'Cancelled', 'No-show')
	reason_for_visit	VARCHAR(255)	—	None
	symptoms	VARCHAR(255)	—	None
	diagnosis	VARCHAR(255)	—	None
	notes	VARCHAR(255)	—	None
	created_by	INT	FK	REFERENCES Admins(admin_id)
	created_at	DATETIME	—	DEFAULT CURRENT_TIMESTAMP
	updated_at	DATETIME	—	DEFAULT CURRENT_TIMESTAMP

Table Name	Column Name	Data Type	Key (PK / FK / —)	Constraints
Admissions	admission_id	INT	PK	PRIMARY KEY, AUTO_INCREMENT
	patient_id	INT	FK	REFERENCES Patients(patient_id)
	doctor_id	INT	FK	REFERENCES Doctors(doctor_id)
	room_id	INT	FK	REFERENCES Rooms(room_id)
	bed_number	VARCHAR(255)	—	None
	admission_date	VARCHAR(255)	—	None
	discharge_date	VARCHAR(255)	—	None
	reason_for_admission	VARCHAR(255)	—	None
	diagnosis	VARCHAR(255)	—	None
	treatment_summary	VARCHAR(255)	—	None
	admission_type	VARCHAR(255)	—	CHECK IN ('Emergency', 'Planned', 'Referral')
	status	VARCHAR(255)	—	CHECK IN ('Admitted', 'Discharged', 'Transferred')
	attendant_name	VARCHAR(255)	—	None
	attendant_phone	VARCHAR(255)	—	None
	created_by	INT	FK	REFERENCES Admins(admin_id)
	created_at	DATETIME	—	DEFAULT CURRENT_TIMESTAMP

Table Name	Column Name	Data Type	Key (PK / FK / —)	Constraints
Rooms	room_id	INT	PK	PRIMARY KEY, AUTO_INCREMENT
	room_number	VARCHAR(255)	—	None
	room_type	VARCHAR(255)	—	CHECK IN ('General','Private', 'Semi-Private','ICU', 'Operation Theatre')
	floor_number	INT	—	None
	department_id	INT	FK	REFERENCES Departments(department_id)
	bed_capacity	INT	—	None
	beds_occupied	INT	—	None
	daily_charge	DECIMAL(10,2)	—	None
	status	VARCHAR(255)	—	CHECK IN ('Available','Occupied', 'Under Maintenance','Reserved')
	created_at	DATETIME	—	DEFAULT CURRENT_TIMESTAMP

Table Name	Column Name	Data Type	Key (PK / FK / —)	Constraints
Prescriptions	prescription_id	INT	PK	PRIMARY KEY, AUTO_INCREMENT
	patient_id	INT	FK	REFERENCES Patients(patient_id)
	doctor_id	INT	FK	REFERENCES Doctors(doctor_id)
	appointment_id	INT	FK	REFERENCES Appointments(appointment_id)
	admission_id	INT	FK	REFERENCES Admissions(admission_id)
	prescription_date	VARCHAR(255)	—	None
	diagnosis	VARCHAR(255)	—	None
	notes	VARCHAR(255)	—	None
	status	VARCHAR(255)	—	CHECK IN ('Active','Completed', 'Cancelled')
	created_at	DATETIME	—	DEFAULT CURRENT_TIMESTAMP

Table Name	Column Name	Data Type	Key (PK / FK / —)	Constraints
Prescription Details	prescription_detail_id	INT	PK	PRIMARY KEY, AUTO_INCREMENT
	prescription_id	INT	FK	REFERENCES Prescriptions(prescription_id)
	medicine_id	INT	FK	REFERENCES Medicines(medicine_id)
	dosage	VARCHAR(255)	—	None
	frequency	VARCHAR(255)	—	None
	duration	VARCHAR(255)	—	None
	instructions	VARCHAR(255)	—	None

Table Name	Column Name	Data Type	Key (PK / FK / —)	Constraints
Medicines	medicine_id	INT	PK	PRIMARY KEY, AUTO_INCREMENT
	medicine_name	VARCHAR(255)	—	None
	generic_name	VARCHAR(255)	—	None
	category	VARCHAR(255)	—	None
	manufacturer	VARCHAR(255)	—	None
	batch_number	VARCHAR(255)	—	None
	unit	VARCHAR(255)	—	None
	stock_quantity	INT	—	None
	reorder_level	INT	—	None
	unit_price	DECIMAL(10,2)	—	None
	expiry_date	VARCHAR(255)	—	None
	manufacture_date	VARCHAR(255)	—	None
	supplier_id	INT	—	None
	status	VARCHAR(255)	—	CHECK IN ('In Stock','Low Stock', 'Out of Stock','Expired')
	created_at	DATETIME	—	DEFAULT CURRENT_TIMESTAMP

Table Name	Column Name	Data Type	Key (PK / FK / —)	Constraints
LabTests	lab_test_id	INT	PK	PRIMARY KEY, AUTO_INCREMENT
	patient_id	INT	FK	REFERENCES Patients(patient_id)
	doctor_id	INT	FK	REFERENCES Doctors(doctor_id)
	test_name	VARCHAR(255)	—	None
	test_category	VARCHAR(255)	—	None
	test_date	VARCHAR(255)	—	None
	sample_collected_date	VARCHAR(255)	—	None
	result	VARCHAR(255)	—	None
	normal_range	VARCHAR(255)	—	None
	report	VARCHAR(255)	—	None
	report_generated_date	VARCHAR(255)	—	None
	report_format	VARCHAR(255)	—	CHECK IN ('PDF','Image','DOCX')
	report_verified_by	INT	FK	REFERENCES Admins(admin_id)
	is_report_ready	INT	—	DEFAULT 0
	status	VARCHAR(255)	—	CHECK IN ('Ordered', 'Sample Collected', 'In Progress', 'Completed', 'Cancelled')
	test_cost	DECIMAL(10,2)	—	None
	technician_id	INT	FK	REFERENCES Admins(admin_id)
	remarks	VARCHAR(255)	—	None
	created_at	DATETIME	—	DEFAULT CURRENT_TIMESTAMP

Table Name	Column Name	Data Type	Key (PK / FK / —)	Constraints
Invoices	invoice_id	INT	PK	PRIMARY KEY, AUTO_INCREMENT
	patient_id	INT	FK	REFERENCES Patients(patient_id)
	appointment_id	INT	FK	REFERENCES Appointments(appointment_id)
	admission_id	INT	FK	REFERENCES Admissions(admission_id)
	invoice_date	VARCHAR(255)	—	None
	consultation_charges	DECIMAL(10,2)	—	None
	room_charges	DECIMAL(10,2)	—	None
	lab_test_charges	DECIMAL(10,2)	—	None
	medicine_charges	DECIMAL(10,2)	—	None
	other_charges	DECIMAL(10,2)	—	None
	discount	DECIMAL(10,2)	—	None
	tax_amount	DECIMAL(10,2)	—	None
	total_amount	DECIMAL(10,2)	—	None
	paid_amount	DECIMAL(10,2)	—	None
	due_amount	DECIMAL(10,2)	—	None
	payment_mode	VARCHAR(255)	—	CHECK IN ('Cash','Card','UPI','Insurance','Online')
	payment_status	VARCHAR(255)	—	CHECK IN ('Paid','Partially Paid','Unpaid','Refunded')
	generated_by	INT	FK	REFERENCES Admins(admin_id)
	created_at	DATETIME	—	DEFAULT CURRENT_TIMESTAMP

Table Name	Column Name	Data Type	Key (PK / FK / —)	Constraints
Admins	admin_id	INT	PK	PRIMARY KEY, AUTO_INCREMENT
	first_name	VARCHAR(255)	—	None
	last_name	VARCHAR(255)	—	None
	username	VARCHAR(255)	—	UNIQUE
	password_hash	VARCHAR(255)	—	None
	email	VARCHAR(255)	—	None
	phone	VARCHAR(255)	—	None
	role	VARCHAR(255)	—	CHECK IN ('SuperAdmin', 'Receptionist', 'Accountant', 'HR', 'Manager')
	department_id	INT	FK	REFERENCES Departments(department_id)
	status	VARCHAR(255)	—	DEFAULT 'Active' CHECK IN ('Active', 'Inactive')
	last_login	VARCHAR(255)	—	None
	created_at	DATETIME	—	DEFAULT CURRENT_TIMESTAMP

Table Name	Column Name	Data Type	Key (PK / FK / —)	Constraints
Inquiries	inquiry_id	INT	PK	PRIMARY KEY, AUTO_INCREMENT
	name	VARCHAR(255)	—	None
	phone	VARCHAR(255)	—	None
	email	VARCHAR(255)	—	None
	inquiry_type	VARCHAR(255)	—	CHECK IN ('General', 'Appointment', 'Billing', 'Complaint', 'Other')
	subject	VARCHAR(255)	—	None
	message	VARCHAR(255)	—	None
	department_id	INT	FK	REFERENCES Departments(department_id)
	assigned_to	INT	FK	REFERENCES Admins(admin_id)
	status	VARCHAR(255)	—	CHECK IN ('Open', 'In Progress', 'Resolved', 'Closed')
	priority	VARCHAR(255)	—	CHECK IN ('Low', 'Medium', 'High')
	response	VARCHAR(255)	—	None
	inquiry_date	VARCHAR(255)	—	None
	resolved_date	VARCHAR(255)	—	None

Table Name	Column Name	Data Type	Key (PK / FK / —)	Constraints
Invoices	invoice_id	INT	PK	PRIMARY KEY, AUTO_INCREMENT
	patient_id	INT	FK	REFERENCES Patients(patient_id)
	appointment_id	INT	FK	REFERENCES Appointments(appointment_id)
	admission_id	INT	FK	REFERENCES Admissions(admission_id)
	invoice_date	VARCHAR(255)	—	None
	consultation_charges	DECIMAL(10,2)	—	None
	room_charges	DECIMAL(10,2)	—	None
	lab_test_charges	DECIMAL(10,2)	—	None
	medicine_charges	DECIMAL(10,2)	—	None
	other_charges	DECIMAL(10,2)	—	None
	discount	DECIMAL(10,2)	—	None
	tax_amount	DECIMAL(10,2)	—	None
	total_amount	DECIMAL(10,2)	—	None
	paid_amount	DECIMAL(10,2)	—	None
	due_amount	DECIMAL(10,2)	—	None
	payment_mode	VARCHAR(255)	—	CHECK IN ('Cash','Card','UPI','Insurance','Online')
	payment_status	VARCHAR(255)	—	CHECK IN ('Paid','Partially Paid','Unpaid','Refunded')
	generated_by	INT	FK	REFERENCES Admins(admin_id)
	created_at	DATETIME	—	DEFAULT CURRENT_TIMESTAMP

Schema Diagram

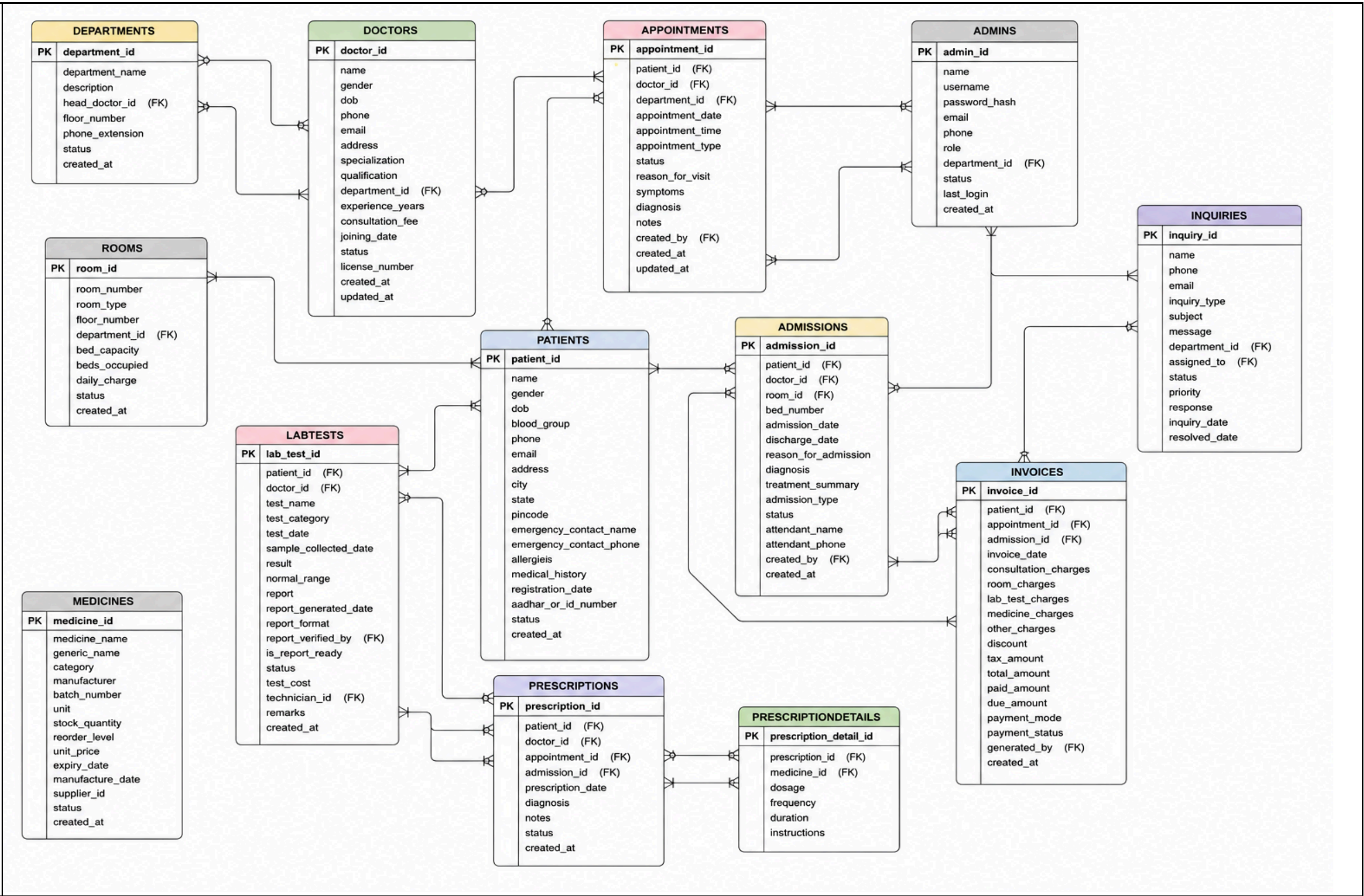


Fig. 3.2: Relational Schema Diagram

3.6 Primary Keys and Foreign Keys

Table 3.5: Primary Key and Foreign Key Definitions

Table	Primary Key	Foreign Key	References (Table.Column)
Admins	admin_id	department_id	Departments.depart ment_id
Admissions	admission_id	patient_id	Patients.patient_id
Appointments	appointment_id	patient_id	Patients.patient_id
Departments	department_id	—	—

Doctors	doctor_id	department_id	Departments.depart ment_id
Inquiries	inquiry_id	department_id	Departments.depart ment_id
Invoices	invoice_id	patient_id	Patients.patient_id
Lab Tests	lab_test_id	patient_id	Patients.patient_id
Medicines	medicine_id	—	—
Patients	patient_id	—	—
Prescription Details	prescription_detail_id	prescription_id	Prescriptions.prescri ption_id
Prescription	prescription_id	patient_id	Patients.patient_id
Rooms	room_id	department_id	Departments.depart ment_id

3.7 Normalization up to 3NF

UNF → 1NF → 2NF → 3NF

Table 3.6: Normalization Steps

Normal Form	Condition Satisfied	Anomaly Removed	Resulting Tables
UNF	Data may contain repeating groups and multiple values in a single record.	— —	Hospital_Records
1NF	All attributes contain atomic/single values and repeating groups are removed.	Repeating-group and insertion anomalies	Patients, Appointments, Admissions
2NF	1NF satisfied and all non-key attributes depend completely on the primary key.	Partial dependency and update anomalies	Patients, Doctors, Departments, Appointments, Admissions, Rooms

3NF	2NF satisfied and non-key attributes depend only on the primary key, with no transitive dependency.	Transitive dependency, update and deletion anomalies	Admins, Patients, Doctors, Departments, Appointments, Admissions, Inquiries, Invoices, Lab Tests, Medicines, Prescription Details, Prescriptions, Rooms
-----	---	--	---

CHAPTER 4: DATABASE IMPLEMENTATION

(Deliverables of Review – II)

4.1 Database and Table Creation – DDL

```
CREATE DATABASE IF NOT EXISTS hospital_db;

USE hospital_db;

SET NAMES utf8mb4;

SET FOREIGN_KEY_CHECKS = 0;

DROP TABLE IF EXISTS `Admins`;

CREATE TABLE `Admins` (
  admin_id INT AUTO_INCREMENT PRIMARY KEY,
```

```
first_name  VARCHAR(255),
last_name   VARCHAR(255),
username    VARCHAR(255) UNIQUE,
password_hash VARCHAR(255),
email       VARCHAR(255),
phone       VARCHAR(255),
role        VARCHAR(255) CHECK(role IN
('SuperAdmin','Receptionist','Accountant','HR','Manager')),
department_id INT,
status      VARCHAR(255) DEFAULT 'Active' CHECK(status IN
('Active','Inactive')),
last_login  VARCHAR(255),
created_at  DATETIME DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (department_id) REFERENCES `Departments`(department_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
DROP TABLE IF EXISTS `Admissions`;
```

```
CREATE TABLE `Admissions` (
  admission_id INT AUTO_INCREMENT PRIMARY KEY,
  patient_id INT,
  doctor_id INT,
  room_id INT,
  bed_number VARCHAR(255),
  admission_date VARCHAR(255),
  discharge_date VARCHAR(255),
  reason_for_admission VARCHAR(255),
  diagnosis VARCHAR(255),
  treatment_summary VARCHAR(255),
  admission_type VARCHAR(255) CHECK(admission_type IN
('Emergency','Planned','Referral')),
  status VARCHAR(255) CHECK(status IN
('Admitted','Discharged','Transferred')),
  attendant_name VARCHAR(255),
  attendant_phone VARCHAR(255),
```

```
created_by      INT,  
created_at      DATETIME DEFAULT CURRENT_TIMESTAMP,  
FOREIGN KEY (patient_id) REFERENCES `Patients`(patient_id),  
FOREIGN KEY (doctor_id) REFERENCES `Doctors`(doctor_id),  
FOREIGN KEY (room_id) REFERENCES `Rooms`(room_id),  
FOREIGN KEY (created_by) REFERENCES `Admins`(admin_id)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
DROP TABLE IF EXISTS `Appointments`;
```

```
CREATE TABLE `Appointments` (  
    appointment_id INT AUTO_INCREMENT PRIMARY KEY,  
    patient_id     INT,  
    doctor_id      INT,  
    department_id  INT,  
    appointment_date VARCHAR(255),  
    appointment_time VARCHAR(255),  
    appointment_type VARCHAR(255) CHECK(appointment_type IN  
(('New','Follow-up'))),  
    status         VARCHAR(255) CHECK(status IN  
(('Scheduled','Completed','Cancelled','No-show'))),  
    reason_for_visit VARCHAR(255),  
    symptoms       VARCHAR(255),  
    diagnosis      VARCHAR(255),  
    notes         VARCHAR(255),  
    created_by     INT,  
    created_at     DATETIME DEFAULT CURRENT_TIMESTAMP,  
    updated_at     DATETIME DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (patient_id) REFERENCES `Patients`(patient_id),  
    FOREIGN KEY (doctor_id) REFERENCES `Doctors`(doctor_id),  
    FOREIGN KEY (department_id) REFERENCES `Departments`(department_id),  
    FOREIGN KEY (created_by) REFERENCES `Admins`(admin_id)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
DROP TABLE IF EXISTS `Departments`;
```

```
CREATE TABLE `Departments` (  
  department_id INT AUTO_INCREMENT PRIMARY KEY,  
  department_name VARCHAR(255) NOT NULL,  
  description VARCHAR(255),  
  head_doctor_id INT,  
  floor_number INT,  
  phone_extension VARCHAR(255),  
  status VARCHAR(255) DEFAULT 'Active' CHECK(status IN  
(('Active','Inactive'))),  
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
DROP TABLE IF EXISTS `Doctors`;
```

```
CREATE TABLE `Doctors` (  
  doctor_id INT AUTO_INCREMENT PRIMARY KEY,  
  first_name VARCHAR(255) NOT NULL,  
  last_name VARCHAR(255) NOT NULL,  
  gender VARCHAR(255) CHECK(gender IN ('M','F','Other')),  
  dob VARCHAR(255),  
  phone VARCHAR(255),  
  email VARCHAR(255) UNIQUE,  
  address VARCHAR(255),  
  specialization VARCHAR(255),  
  qualification VARCHAR(255),  
  department_id INT,  
  experience_years INT,  
  consultation_fee DECIMAL(10,2),  
  joining_date VARCHAR(255),  
  status VARCHAR(255) DEFAULT 'Active' CHECK(status IN  
(('Active','Inactive','On Leave'))),  
  license_number VARCHAR(255) UNIQUE,  
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP,
```

```
FOREIGN KEY (department_id) REFERENCES `Departments`(department_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
DROP TABLE IF EXISTS `Inquiries`;
```

```
CREATE TABLE `Inquiries` (
  inquiry_id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(255),
  phone VARCHAR(255),
  email VARCHAR(255),
  inquiry_type VARCHAR(255) CHECK(inquiry_type IN
('General','Appointment','Billing','Complaint','Other')),
  subject VARCHAR(255),
  message VARCHAR(255),
  department_id INT,
  assigned_to INT,
  status VARCHAR(255) CHECK(status IN ('Open','In
Progress','Resolved','Closed')),
  priority VARCHAR(255) CHECK(priority IN ('Low','Medium','High')),
  response VARCHAR(255),
  inquiry_date VARCHAR(255),
  resolved_date VARCHAR(255),
  FOREIGN KEY (department_id) REFERENCES `Departments`(department_id),
  FOREIGN KEY (assigned_to) REFERENCES `Admins`(admin_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
DROP TABLE IF EXISTS `Invoices`;
```

```
CREATE TABLE `Invoices` (
  invoice_id INT AUTO_INCREMENT PRIMARY KEY,
  patient_id INT,
  appointment_id INT,
  admission_id INT,
  invoice_date VARCHAR(255),
  consultation_charges DECIMAL(10,2),
```

```

room_charges      DECIMAL(10,2),
lab_test_charges   DECIMAL(10,2),
medicine_charges   DECIMAL(10,2),
other_charges      DECIMAL(10,2),
discount           DECIMAL(10,2),
tax_amount         DECIMAL(10,2),
total_amount       DECIMAL(10,2),
paid_amount        DECIMAL(10,2),
due_amount         DECIMAL(10,2),
payment_mode       VARCHAR(255) CHECK(payment_mode IN
('Cash','Card','UPI','Insurance','Online')),
payment_status     VARCHAR(255) CHECK(payment_status IN ('Paid','Partially
Paid','Unpaid','Refunded')),
generated_by       INT,
created_at         DATETIME DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (patient_id) REFERENCES `Patients`(patient_id),
FOREIGN KEY (appointment_id) REFERENCES `Appointments`(appointment_id),
FOREIGN KEY (admission_id) REFERENCES `Admissions`(admission_id),
FOREIGN KEY (generated_by) REFERENCES `Admins`(admin_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

```

DROP TABLE IF EXISTS `LabTests`;

```

```

CREATE TABLE `LabTests` (
  lab_test_id      INT AUTO_INCREMENT PRIMARY KEY,
  patient_id       INT,
  doctor_id        INT,
  test_name        VARCHAR(255),
  test_category    VARCHAR(255),
  test_date        VARCHAR(255),
  sample_collected_date VARCHAR(255),
  result           VARCHAR(255),
  normal_range     VARCHAR(255),
  report           VARCHAR(255),
  report_generated_date VARCHAR(255),

```

```
report_format    VARCHAR(255) CHECK(report_format IN
('PDF','Image','DOCX')),
report_verified_by  INT,
is_report_ready    INT DEFAULT 0,
status            VARCHAR(255) CHECK(status IN ('Ordered','Sample Collected','In
Progress','Completed','Cancelled')),
test_cost         DECIMAL(10,2),
technician_id     INT,
remarks          VARCHAR(255),
created_at        DATETIME DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (patient_id) REFERENCES `Patients`(patient_id),
FOREIGN KEY (doctor_id) REFERENCES `Doctors`(doctor_id),
FOREIGN KEY (report_verified_by) REFERENCES `Admins`(admin_id),
FOREIGN KEY (technician_id) REFERENCES `Admins`(admin_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
DROP TABLE IF EXISTS `Medicines`;
```

```
CREATE TABLE `Medicines` (
  medicine_id    INT AUTO_INCREMENT PRIMARY KEY,
  medicine_name  VARCHAR(255),
  generic_name   VARCHAR(255),
  category      VARCHAR(255),
  manufacturer   VARCHAR(255),
  batch_number  VARCHAR(255),
  unit          VARCHAR(255),
  stock_quantity INT,
  reorder_level INT,
  unit_price    DECIMAL(10,2),
  expiry_date   VARCHAR(255),
  manufacture_date VARCHAR(255),
  supplier_id   INT,
  status        VARCHAR(255) CHECK(status IN ('In Stock','Low Stock','Out of
Stock','Expired')),
  created_at    DATETIME DEFAULT CURRENT_TIMESTAMP
```

) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

DROP TABLE IF EXISTS `Patients`;

CREATE TABLE `Patients` (

patient_id INT AUTO_INCREMENT PRIMARY KEY,
first_name VARCHAR(255) NOT NULL,
last_name VARCHAR(255) NOT NULL,
gender VARCHAR(255) CHECK(gender IN ('M','F','Other')),
dob VARCHAR(255),
blood_group VARCHAR(255),
phone VARCHAR(255),
email VARCHAR(255),
address VARCHAR(255),
city VARCHAR(255),
state VARCHAR(255),
pincode VARCHAR(255),
emergency_contact_name VARCHAR(255),
emergency_contact_phone VARCHAR(255),
allergies VARCHAR(255),
medical_history VARCHAR(255),
registration_date VARCHAR(255),
aadhar_or_id_number VARCHAR(255),
status VARCHAR(255) DEFAULT 'Active' CHECK(status IN
('Active','Inactive','Deceased')),
created_at DATETIME DEFAULT CURRENT_TIMESTAMP

) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

DROP TABLE IF EXISTS `PrescriptionDetails`;

CREATE TABLE `PrescriptionDetails` (

prescription_detail_id INT AUTO_INCREMENT PRIMARY KEY,
prescription_id INT,
medicine_id INT,
dosage VARCHAR(255),

```
frequency      VARCHAR(255),
duration       VARCHAR(255),
instructions    VARCHAR(255),
FOREIGN KEY (prescription_id) REFERENCES `Prescriptions`(prescription_id),
FOREIGN KEY (medicine_id) REFERENCES `Medicines`(medicine_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

DROP TABLE IF EXISTS `Prescriptions`;

CREATE TABLE `Prescriptions` (
  prescription_id INT AUTO_INCREMENT PRIMARY KEY,
  patient_id      INT,
  doctor_id       INT,
  appointment_id  INT,
  admission_id    INT,
  prescription_date VARCHAR(255),
  diagnosis       VARCHAR(255),
  notes          VARCHAR(255),
  status          VARCHAR(255) CHECK(status IN ('Active','Completed','Cancelled')),
  created_at      DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (patient_id) REFERENCES `Patients`(patient_id),
  FOREIGN KEY (doctor_id) REFERENCES `Doctors`(doctor_id),
  FOREIGN KEY (appointment_id) REFERENCES `Appointments`(appointment_id),
  FOREIGN KEY (admission_id) REFERENCES `Admissions`(admission_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

DROP TABLE IF EXISTS `Rooms`;

CREATE TABLE `Rooms` (
  room_id      INT AUTO_INCREMENT PRIMARY KEY,
  room_number  VARCHAR(255),
  room_type    VARCHAR(255) CHECK(room_type IN
('General','Private','Semi-Private','ICU','Operation Theatre')),
  floor_number INT,
  department_id INT,
```

```
bed_capacity INT,  
beds_occupied INT,  
daily_charge DECIMAL(10,2),  
status VARCHAR(255) CHECK(status IN ('Available','Occupied','Under  
Maintenance','Reserved')),  
created_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
FOREIGN KEY (department_id) REFERENCES `Departments`(department_id)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;  
  
SET FOREIGN_KEY_CHECKS = 1;
```

	doctor_id	first_name	last_name	specialization	experience_years
▶	2	Liam	Dugal	Epileptology	15
	3	Liam	Setty	Sports Medicine	24
	5	Sai	Sidhu	General Dermatology	25
	6	Ojasvi	Suresh	Head & Neck Surgery	20
	7	Dipta	Kota	Internal Medicine	29
	8	Vritti	Ray	High-risk Obstetrics	13
	9	Chandran	Bedi	Medical Oncology	19
	10	Azaan	Murthy	Child Psychiatry	28
	11	Ekaja	Mital	Interv	Interventional Cardiology
	12	Ekani	Lalla	Stroke medicine	
	13	Dayita	Borra	Joint Replacement	13
	14	Ojas	Sabharwal	General Pediatrics	6
	15	Devika	Lal	General Dermatology	18
	17	Yasti	Nayak	General Physician	26
	18	Vincent	Bedi	General Gynecology	13

	patient_id	first_name	last_name	gender	blood_group	phone
	1	Ekbal	Kata	M	B+	04740748217
	2	Ladli	Devan	F	A+	1047095214
	3	Rayaan	Prabhu	M	A-	+913709859317
	4	Kamya	Patil	F	O+	+915850643171
	5	Ojasvi	Babu	F	O-	+914026811775
	6	Vasana	Tiwari	F	A-	6118367365
	7	Mitali	Muni	F	O+	01983273158
	9	Tanveer	Gill	M	B+	+916838851606
	10	Ojas	Ganguly	M	A+	+918188355231
	11	Anirudh	Bhandari	M	B-	04119986798
	12	Oviya	Bhatti	F	A+	01864492519
	13	Daksha	Tripathi	F	O-	8059296222
	14	Dalaja	Varty	F	B-	+914124782613
	15	Falan	Wable	M	O+	01434103697
	16	Anthony	Ram	M	AB+	04051531952

Fig. 4.1: Successful Creation of Tables

4.2 Sample Data Insertion – DML

Table 4.1: Sample Data Summary

S.No	Table Name	No. of Records Inserted	Screenshot Reference
1	Departments	10	Figure 4.1
2	Admins	10	Figure 4.1
3	Doctors	30	Figure 4.1
4	Patients	100	Figure 4.1
5	Rooms	50	Figure 4.1
6	Medicines	40	Figure 4.1
7	Appointments	180	Figure 4.1
8	Admissions	30	Figure 4.1
9	Prescriptions	90	Figure 4.1
10	Prescription Details	226	Figure 4.1
11	Lab Tests	110	Figure 4.1
12	Invoices	130	Figure 4.1
13	Inquiries	40	Figure 4.1

```
INSERT INTO `Admins`
VALUES(1,'Radhika','Tara','radhikatara0','f375a538896767e752c75e0bc1028cbb5
9b1459bd7b2d9b5c220f00ef1a07057','radhika.tara0@cityhospital.com','+913482
477109','SuperAdmin',NULL,'Active','2026-04-10 09:39:20','2026-08-12 08:44:07');

INSERT INTO `Admins`
VALUES(2,'Kashvi','Dutt','kashvidutt1','fd48ded432917e368ab7bfc10dd7bce1816b9
55f8dc0e821fd4969a039d37f43','kashvi.dutt1@cityhospital.com','+918613171274',
'HR',9,'Active','2026-06-19 17:43:08','2026-08-12 08:44:07');

INSERT INTO `Admins`
VALUES(3,'Laban','Sagar','labansagar2','7d9a2b8f9316a8ef1f50168684d861d8095
```

e040bda2fc505e21188f02e3aa760','laban.sagar2@cityhospital.com','03782639821','Accountant',NULL,'Active','2026-01-15 21:05:14','2026-08-12 08:44:07');

INSERT INTO `Admins`

VALUES(4,'Lakshit','Sachdeva','lakshitsachdeva3','8f582f7927625174fec5e79f3764c1ae6b98546d0f20d9f300cf25a79d2942db','lakshit.sachdeva3@cityhospital.com','4044997278','HR',NULL,'Active','2026-05-30 17:21:33','2026-08-12 08:44:07');

INSERT INTO `Admins`

VALUES(5,'Wyatt','Pai','wyattpai4','994179063a51364c111145ecf9946dfdcd11042701305b72db3a431a1550c5b9','wyatt.pai4@cityhospital.com','6753396360','Receptionist',2,'Active','2026-08-06 06:45:56','2026-08-12 08:44:07');

INSERT INTO `Admins`

VALUES(6,'Nathan','Solanki','nathansolanki5','f152051026c302ee6f8f3af3284d8d046e04cbd88e624d57fcbe87d7968d2766','nathan.solanki5@cityhospital.com','06270289517','Receptionist',4,'Active','2026-04-14 04:21:53','2026-08-12 08:44:07');

INSERT INTO `Admins`

VALUES(7,'Chandresh','Vora','chandreshvora6','03f5003e1967ac582cbf5a35125116191106c601f5d93389ed9603c9cfb568e3','chandresh.vora6@cityhospital.com','+912621745961','Receptionist',7,'Active','2026-06-18 21:15:03','2026-08-12 08:44:07');

INSERT INTO `Admins`

VALUES(8,'Omya','Walla','omyawalla7','c45641253680ce050ab6a48cb7d990af651e6beb9aaeca7d05feaba61e8e3086','omya.walla7@cityhospital.com','07809134316','HR',NULL,'Active','2026-07-03 11:26:20','2026-08-12 08:44:07');

INSERT INTO `Admins`

VALUES(9,'Chandran','Chaudhry','chandranchaudhry8','e343e521c4f1b214cb0ca41b1ce810108d2763103607d4394a255d1d21c54912','chandran.chaudhry8@cityhospital.com','4005045562','HR',5,'Active','2026-04-28 11:30:41','2026-08-12 08:44:07');

**INSERT INTO `Admins`
VALUES(10,'Jason','Wali','jasonwali9','3aeba03c25b583f2f78931583766bcbf699a2156c5d198090ca261ae2632ac63','jason.wali9@cityhospital.com','2221969379','Receptionist',10,'Active','2026-01-26 02:40:02','2026-08-12 08:44:07');**

1101 10:09:34 INSERT INTO 'Admins' VALUES(1,'Radhika','Tara','radhikatar0','f375a538896767e752c... 1 row(s) affected 0.015 sec

	admin_id	first_name	last_name	username	password_hash	email	phone	role	department_id	sta
▶	1	Radhika	Tara	radhikatar0	f375a538896767e752c75e0bc1028cbb59b1459...	radhika.tara0@cityhospital.com	+913482477109	SuperAdmin	NULL	Acti
	2	Kashvi	Dutt	kashvidutt1	fd48ded432917e368ab7bfc10dd7bce1816b955...	kashvi.dutt1@cityhospital.com	+918613171274	HR	9	Acti
	3	Laban	Sagar	labansagar2	7d9a2b8f9316a8ef1f50168684d861d8095e040...	laban.sagar2@cityhospital.com	03782639821	Accountant	NULL	Acti
	4	Lakshit	Sachdeva	lakshitsachdeva3	8f582f7927625174fec5e79f3764c1ae6b98546d...	lakshit.sachdeva3@cityhospital.com	4044997278	HR	NULL	Acti
	5	Wyatt	Pai	wyattpai4	994179063a51364c111145ecf9946dfdcd11042...	wyatt.pai4@cityhospital.com	6753396360	Receptionist	2	Acti
	6	Nathan	Solanki	nathansolanki5	f152051026c302ee6f8f3af3284d8d046e04cbd8...	nathan.solanki5@cityhospital.com	06270289517	Receptionist	4	Acti
	7	Chandresh	Vora	chandreshvora6	03f5003e1967ac582cbf5a35125116191106c60...	chandresh.vora6@cityhospital.com	+912621745961	Receptionist	7	Acti
	8	Omya	Walla	omyawalla7	c45641253680ce050ab6a48cb7d990af651e6be...	omya.walla7@cityhospital.com	07809134316	HR	NULL	Acti
	9	Chandran	Chaudhry	chandranchaudhry8	e343e521c4f1b214cb0ca41b1ce810108d27631...	chandran.chaudhry8@cityhospital.com	4005045562	HR	5	Acti
	10	Jason	Wali	jasonwali9	3aeba03c25b583f2f78931583766bcbf699a215...	jason.wali9@cityhospital.com	2221969379	Receptionist	10	Acti
•	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Fig. 4.2: Sample Data Inserted into the Tables

4.3 Constraints and Data Integrity

Table 4.2: Constraints and Data Integrity Rules

Constraint	Applied On	Purpose	Evidence
PRIMARY KEY with AUTO_INCREMENT	The ID column of all 13 tables (e.g., patient_id, doctor_id, invoice_id)	Gives every record a unique, non-null identity and generates IDs automatically (entity integrity)	Figure 8.1: duplicate ID rejected (Error 1062)
FOREIGN KEY	27 columns, e.g., Appointments.patient_id → Patients, Doctors.department_id → Departments, Invoices.admission_id → Admissions	Keeps links between tables valid: no record without an existing parent, and no parent deleted while child records exist (referential integrity)	Figure 8.2: non-existent patient rejected (Error 1452) Figure 8.3: deleting a patient who has appointments rejected (Error 1451)
UNIQUE	Doctors.email, Doctors.license_number, Admins.username	Prevents duplicate emails, licence numbers and login names	Figure 8.4: duplicate licence number rejected (Error 1062)
NOT NULL	Patients.first_name and last_name, Doctors.first_name and last_name, Departments.department_name	Ensures essential identifying fields are never left empty	Figure 8.5: NULL first name rejected (Error 1048)
CHECK	22 columns, e.g., gender ('M','F','Other'), appointment status, room_type, payment_status, Admins.role	Restricts a column to its allowed values (domain integrity)	Figure 8.6: invalid gender 'X' rejected (Error 3819)
DEFAULT	status = 'Active' (Patients, Doctors, Departments, Admins); created_at and updated_at = CURRENT_TIMESTAMP; is_report_ready = 0 (LabTests)	Fills sensible values automatically when none are given	Figure 8.7: new department gets status 'Active' and a timestamp
Data types	INT, VARCHAR(255), DECIMAL(10,2), DATETIME across all tables	Limits each column to the right kind of data; money is held to 2 decimals	Table 3.4 (Relational Database Schema)

4.4 Data Validation Rules

Explain how duplicate IDs, invalid values and orphan records are prevented. Demonstrate one referential integrity violation and the error the database returns.

Data validation is implemented using database constraints. Primary keys and AUTO_INCREMENT prevent duplicate IDs in each table. UNIQUE constraints prevent duplicate values such as usernames, email addresses and license numbers. CHECK constraints restrict fields to valid predefined values, such as admission type, status, gender and payment mode. Foreign key constraints maintain referential integrity by ensuring that referenced records exist in the parent tables. Therefore, invalid values, duplicate identifiers and orphan records are prevented by the database.



2187 10:30:51 INSERT INTO Admissions (patient_id, doctor_id, room_id, admission_type, status) VALUE... Error Code: 1452. Cannot add or update a child row: a foreign key constraint fails (hospital... 0.016 sec

Fig. 4.3: Referential Integrity Enforced by the Database

CHAPTER 5: SQL OPERATIONS AND QUERIES

For every query, give the purpose, the SQL statement and the output. Number them continuously (Q1, Q2, ...). A minimum of fifteen queries covering all the sub-sections below is expected.

5.1 DQL – Basic Retrieval Queries

Table 5.1: Basic Retrieval Queries

Q.No	Purpose of the Query	SQL Statement	Output Ref.
Q1	Display all departments with their floor number and phone extension	<pre>SELECT department_id, department_name, floor_number, phone_extension FROM Departments;</pre>	Fig. 5.1
Q2	List active doctors having more than 20 years of experience	<pre>SELECT doctor_id, first_name, last_name, specialization, experience_years FROM Doctors WHERE status = 'Active' AND experience_years > 20 ORDER BY doctor_id;</pre>	Fig. 5.2
Q3	Show the five highest consultation fees	<pre>SELECT doctor_id, first_name, last_name, consultation_fee FROM Doctors ORDER BY consultation_fee DESC, doctor_id LIMIT 5;</pre>	Fig. 5.3
Q4	Find Andhra Pradesh patients whose first name starts with 'R'	<pre>SELECT patient_id, first_name, last_name, city FROM Patients WHERE state = 'Andhra Pradesh' AND first_name LIKE 'R%' ORDER BY patient_id;</pre>	Fig. 5.4
Q5	List the distinct cities of patients from Andhra Pradesh and Telangana	<pre>SELECT DISTINCT city FROM Patients WHERE state IN ('Andhra Pradesh', 'Telangana') ORDER BY city;</pre>	Fig. 5.5
Q6	Display available rooms whose daily charge is between 5000 and 8000	<pre>SELECT room_id, room_number, room_type, daily_charge FROM Rooms WHERE daily_charge BETWEEN 5000 AND 8000 AND status = 'Available' ORDER BY room_id;</pre>	Fig. 5.6

5.2 Join Operations

Include natural join, equi join, inner join, left outer, right outer and full outer join.

Table 5.2: Join Operations

Q.No	Type of Join	SQL Statement	Output Ref.
Q7	Natural Join	<pre>SELECT prescription_detail_id, prescription_id, medicine_name, dosage, frequency FROM PrescriptionDetails NATURAL JOIN Medicines ORDER BY prescription_detail_id LIMIT 5;</pre>	Fig. 5.7
Q8	Equi Join	<pre>SELECT d.doctor_id, d.first_name, d.last_name, dp.department_name FROM Doctors d, Departments dp WHERE d.department_id = dp.department_id ORDER BY d.doctor_id LIMIT 6;</pre>	Fig. 5.8
Q9	Inner Join	<pre>SELECT a.appointment_id, p.first_name AS patient, d.last_name AS doctor, a.appointment_date, a.status FROM Appointments a INNER JOIN Patients p ON a.patient_id = p.patient_id INNER JOIN Doctors d ON a.doctor_id = d.doctor_id WHERE a.status = 'Completed' ORDER BY a.appointment_id LIMIT 6;</pre>	Fig. 5.9
Q10	Left Outer Join	<pre>SELECT p.patient_id, p.first_name, a.admission_id, a.admission_date FROM Patients p LEFT OUTER JOIN Admissions a ON p.patient_id = a.patient_id WHERE p.patient_id BETWEEN 11 AND 16 ORDER BY p.patient_id, a.admission_id;</pre>	Fig. 5.10
Q11	Right Outer Join	<pre>SELECT a.admission_id, a.admission_date, r.room_number, r.room_type FROM Admissions a RIGHT OUTER JOIN Rooms r ON a.room_id = r.room_id ORDER BY r.room_id, a.admission_id LIMIT 8;</pre>	Fig. 5.11
Q12	Full Outer Join (emulated in MySQL as LEFT JOIN UNION RIGHT JOIN)	<pre>SELECT d.department_name, a.admin_id, a.first_name, a.role FROM Departments d LEFT JOIN Admins a ON d.department_id = a.department_id UNION SELECT d.department_name, a.admin_id, a.first_name, a.role FROM Departments d RIGHT JOIN Admins a ON d.department_id = a.department_id ORDER BY department_name, admin_id;</pre>	Fig. 5.12

5.3 Aggregate Functions, GROUP BY and HAVING

Table 5.3: Aggregate, GROUP BY and HAVING Queries

Q.No	Function Used	SQL Statement	Output Ref.
Q13	COUNT() with GROUP BY	SELECT state, COUNT(*) AS total_patients FROM Patients GROUP BY state ORDER BY total_patients DESC;	Fig. 5.13
Q14	SUM() and COUNT() with GROUP BY	SELECT payment_status, COUNT(*) AS invoices, ROUND(SUM(total_amount), 2) AS total_billed, ROUND(SUM(paid_amount), 2) AS total_paid FROM Invoices GROUP BY payment_status ORDER BY payment_status;	Fig. 5.14
Q15	AVG(), MIN() and MAX() with GROUP BY	SELECT dp.department_name, ROUND(AVG(d.consultation_fee), 2) AS avg_fee, MIN(d.consultation_fee) AS min_fee, MAX(d.consultation_fee) AS max_fee FROM Doctors d JOIN Departments dp ON d.department_id = dp.department_id GROUP BY dp.department_name ORDER BY dp.department_name;	Fig. 5.15
Q16	COUNT() with WHERE, GROUP BY and HAVING	SELECT doctor_id, COUNT(*) AS completed_appts FROM Appointments WHERE status = 'Completed' GROUP BY doctor_id HAVING COUNT(*) >= 6 ORDER BY completed_appts DESC, doctor_id;	Fig. 5.16
Q17	COUNT(), MAX() and SUM() with HAVING	SELECT room_type, COUNT(*) AS total_rooms, MAX(daily_charge) AS max_charge, SUM(bed_capacity) AS total_beds FROM Rooms GROUP BY room_type HAVING COUNT(*) > 10 ORDER BY total_rooms DESC, room_type;	Fig. 5.17

5.4 Nested and Correlated Subqueries

Table 5.4: Subqueries

Q.No	Type (Nested / Correlated)	SQL Statement	Output Ref.
Q18	Nested (single-row / scalar subquery)	<pre>SELECT doctor_id, first_name, last_name, consultation_fee FROM Doctors WHERE consultation_fee > (SELECT AVG(consultation_fee) FROM Doctors) ORDER BY consultation_fee DESC, doctor_id;</pre>	Fig. 5.18
Q19	Nested (multi-row, IN, two levels)	<pre>SELECT patient_id, first_name, last_name FROM Patients WHERE patient_id IN (SELECT patient_id FROM Admissions WHERE room_id IN (SELECT room_id FROM Rooms WHERE room_type = 'ICU')) ORDER BY patient_id;</pre>	Fig. 5.19
Q20	Nested (multi-row, NOT IN)	<pre>SELECT doctor_id, first_name, last_name FROM Doctors WHERE doctor_id NOT IN (SELECT doctor_id FROM Admissions) ORDER BY doctor_id;</pre>	Fig. 5.20
Q21	Correlated (comparison with the outer row)	<pre>SELECT d.doctor_id, d.first_name, d.last_name, d.department_id, d.consultation_fee FROM Doctors d WHERE d.consultation_fee > (SELECT AVG(d2.consultation_fee) FROM Doctors d2 WHERE d2.department_id = d.department_id) ORDER BY d.doctor_id;</pre>	Fig. 5.21
Q22	Correlated (EXISTS)	<pre>SELECT p.patient_id, p.first_name, p.last_name FROM Patients p WHERE EXISTS (SELECT 1 FROM Appointments a WHERE a.patient_id = p.patient_id AND a.status = 'No-show') ORDER BY p.patient_id;</pre>	Fig. 5.22

5.5 Views and Relational Set Operations

Include at least one updatable view and one non-updatable view, and the set operations UNION, INTERSECT and MINUS / EXCEPT.

Table 5.5: Views and Set Operations

Q.No	View / Set Operation	SQL Statement	Output Ref.
Q23	Updatable View (simple view on a single table)	<pre>CREATE VIEW vw_active_doctors AS SELECT doctor_id, first_name, last_name, specialization, consultation_fee FROM Doctors WHERE status = 'Active'; UPDATE vw_active_doctors SET consultation_fee = 850 WHERE doctor_id = 1; SELECT * FROM vw_active_doctors WHERE doctor_id = 1;</pre>	Fig. 5.23
Q24	Non-Updatable View (uses GROUP BY and aggregate functions)	<pre>CREATE VIEW vw_dept_fee_summary AS SELECT department_id, COUNT(*) AS total_doctors, ROUND(AVG(consultation_fee), 2) AS avg_fee FROM Doctors GROUP BY department_id; SELECT * FROM vw_dept_fee_summary ORDER BY department_id; UPDATE vw_dept_fee_summary SET total_doctors = 5 WHERE department_id = 1;</pre>	Fig. 5.24
Q25	UNION	<pre>SELECT first_name, last_name, 'Doctor' AS staff_type FROM Doctors WHERE department_id = 1 UNION SELECT first_name, last_name, 'Admin' FROM Admins WHERE role = 'HR' ORDER BY last_name;</pre>	Fig. 5.25
Q26	INTERSECT	<pre>SELECT patient_id FROM Appointments WHERE status = 'No-show' INTERSECT SELECT patient_id FROM Invoices WHERE payment_status = 'Unpaid' ORDER BY patient_id;</pre>	Fig. 5.26
Q27	EXCEPT (MINUS in Oracle)	<pre>SELECT patient_id FROM Appointments WHERE status = 'No-show' EXCEPT SELECT patient_id FROM Invoices WHERE payment_status = 'Unpaid' ORDER BY patient_id;</pre>	Fig. 5.27

5.6 DCL and TCL Operations

Demonstrate GRANT and REVOKE (DCL), and COMMIT, SAVEPOINT and ROLLBACK (TCL).

Table 5.6: DCL and TCL Operations

Q.No	Command	Purpose	SQL Statement	Output Ref.
Q28	GRANT	Create a user and give it SELECT and INSERT rights on the Patients table	<pre>CREATE USER 'reception_user'@'localhost' IDENTIFIED BY 'Recep@123'; GRANT SELECT, INSERT ON hospital_db.Patients TO 'reception_user'@'localhost'; SHOW GRANTS FOR 'reception_user'@'localhost';</pre>	Fig. 5.28
Q29	REVOKE	Withdraw the INSERT privilege from the user	<pre>REVOKE INSERT ON hospital_db.Patients FROM 'reception_user'@'localhost'; SHOW GRANTS FOR 'reception_user'@'localhost';</pre>	Fig. 5.29
Q30	COMMIT	Make an update permanent by ending the transaction	<pre>START TRANSACTION; UPDATE Rooms SET status = 'Under Maintenance' WHERE room_id = 3; COMMIT; SELECT room_id, room_number, room_type, status FROM Rooms WHERE room_id = 3;</pre>	Fig. 5.30
Q31	SAVEPOINT	Mark a point inside a transaction to which we can return later	<pre>START TRANSACTION; UPDATE Rooms SET daily_charge = daily_charge + 500 WHERE room_id = 2; SAVEPOINT after_icu; UPDATE Rooms SET daily_charge = daily_charge + 200 WHERE room_id = 7; SELECT room_id, room_type, daily_charge FROM Rooms WHERE room_id IN (2, 7);</pre>	Fig. 5.31
Q32	ROLLBACK TO SAVEPOINT	Undo only the changes made after the savepoint	<pre>ROLLBACK TO SAVEPOINT after_icu; SELECT room_id, room_type, daily_charge FROM Rooms WHERE room_id IN (2, 7);</pre>	Fig. 5.32
Q33	ROLLBACK	Cancel the whole transaction and restore the original values	<pre>ROLLBACK; SELECT room_id, room_type, daily_charge FROM Rooms WHERE room_id IN (2, 7);</pre>	Fig. 5.33

CHAPTER 6: MODULES AND FUNCTIONALITIES

Explain the major modules of the system, what each one does and which tables it operates on.

Table 6.1: Modules and Their Functionalities

S.No	Module Name	Functionality	Tables Involved
1	Administration & User Management	Creates staff accounts with role-based access (SuperAdmin, Receptionist, Accountant, HR, Manager), stores passwords only as hashes and tracks account status and last login.	Admins, Departments
2	Patient Management	Registers patients and maintains their personal details, blood group, allergies, medical history, emergency contact and status (Active / Inactive / Deceased).	Patients
3	Doctor & Department Management	Maintains doctor profiles (specialization, qualification, licence number, experience, consultation fee, availability) and hospital departments with head doctor, floor and phone extension.	Doctors, Departments
4	Appointment Management	Books New and Follow-up appointments between patients and doctors, tracks their status (Scheduled, Completed, Cancelled, No-show) and records symptoms, diagnosis and notes.	Appointments, Patients, Doctors, Departments, Admins
5	Admission & Room Management	Admits patients (Emergency, Planned, Referral), allocates rooms and beds, tracks bed occupancy and daily room charge, and records diagnosis, treatment summary and discharge or transfer.	Admissions, Rooms, Patients, Doctors, Departments
6	Prescription Management	Records the diagnosis and the medicines prescribed by a doctor for an appointment or an admission, with dosage, frequency, duration and instructions for each medicine.	Prescriptions, PrescriptionDetails, Medicines, Patients, Doctors, Appointments, Admissions
7	Medicine Inventory Management	Maintains the medicine catalogue with batch number, stock quantity, reorder level, unit price, manufacture and expiry dates, and stock status (In Stock, Low Stock, Out of Stock, Expired).	Medicines, PrescriptionDetails
8	Laboratory Management	Orders lab tests for patients, tracks sample collection, results against the normal range, report format and verification, and the test status and cost.	LabTests, Patients, Doctors, Admins
9	Billing & Invoice Management	Generates invoices from consultation, room, lab, medicine and other charges with discount and tax, and records paid amount, due amount, payment mode and payment status.	Invoices, Patients, Appointments, Admissions, Admins
10	Inquiry & Support Management	Logs inquiries (General, Appointment, Billing, Complaint, Other), assigns them to staff with a priority and tracks the response until they are resolved or closed.	Inquiries, Departments, Admins

1. Administration&User Management. This module controls who can use the system. Every administrator has a unique username, a role and an Active/Inactive status, and passwords are kept only as hashes. The role decides which of the other modules a staff member works in, and the administrator's ID is stored as the creator of appointments, admissions and invoices so that every action can be traced.

2. Patient Management. This module is the starting point for all clinical work. A patient is registered once with demographic and contact details, blood group, allergies and past medical history, and every later appointment, admission, prescription, lab test and invoice refers back to that single patient record.

3. Doctor&Department Management. This module stores who treats patients and where they work. Each doctor belongs to a department, has a unique licence number and e-mail, and carries a consultation fee and a status of Active, Inactive or On Leave. Departments record their head doctor, floor number and phone extension.

4. Appointment Management. This module handles outpatient visits. Staff book an appointment for a patient with a doctor of a department on a given date and time, and

the status is updated as the visit progresses. After the consultation, the symptoms, diagnosis and notes are stored against the same appointment.

5. Admission&Room Management. This module manages inpatient care. Rooms are classified as General, Private, Semi-Private, ICU or Operation Theatre, with bed capacity, occupied beds and a daily charge. When a patient is admitted, a room and bed are assigned, and the admission stays open until the patient is discharged or transferred.

6. Prescription Management. This module records what the doctor prescribes. A prescription is linked to a patient, a doctor and either an appointment or an admission, and each medicine on it is stored as a separate line in PrescriptionDetails with its dosage, frequency and duration. The status shows whether it is Active, Completed or Cancelled.

7. Medicine Inventory Management. This module keeps the pharmacy stock under control. Every medicine has a batch number, quantity in stock, a reorder level and an expiry date, so low or expired stock can be identified quickly. The link with PrescriptionDetails shows which medicines are actually being prescribed.

8. Laboratory Management. This module covers diagnostic testing. A doctor orders a test for a patient, a technician collects the sample and enters the result along with its normal range, and the report is verified by an administrator. The status moves from Ordered through Sample Collected and In Progress to Completed, or to Cancelled.

9. Billing&Invoice Management. This module produces the bill for a patient. An invoice is tied to an appointment or an admission and adds up the consultation, room, laboratory, medicine and other charges, subtracts any discount and adds tax. It also stores the amount paid, the amount still due, the payment mode and the payment status.

10. Inquiry&Support Management. This module handles questions and complaints from the public. Each inquiry is recorded with its type, subject, message and priority, optionally linked to a department and assigned to an administrator. It stays Open or In Progress until a response is given and it is marked Resolved or Closed.

CHAPTER 7: RESULTS - SAMPLE OUTPUTS AND SCREENSHOTS

Paste the important execution screenshots. Each screenshot must clearly show both the query and its output, and must carry a numbered caption.

SELECT patient_id, first_name, last_name, gender, blood_group, phone
FROM Patients
WHERE status = 'Active';

	patient_id	first_name	last_name	gender	blood_group	phone
	1	Ekbal	Kata	M	B+	04740748217
	2	Ladli	Devan	F	A+	1047095214
	3	Rayaan	Prabhu	M	A-	+913709859317
	4	Kamya	Patil	F	O+	+915850643171
	5	Ojasvi	Babu	F	O-	+914026811775
	6	Vasana	Tiwari	F	A-	6118367365
	7	Mitali	Muni	F	O+	01983273158
	9	Tanveer	Gill	M	B+	+916838851606
	10	Ojas	Ganguly	M	A+	+918188355231
	11	Anirudh	Bhandari	M	B-	04119986798
	12	Oviya	Bhatti	F	A+	01864492519
	13	Daksha	Tripathi	F	O-	8059296222
	14	Dalaja	Varty	F	B-	+914124782613
	15	Falan	Wable	M	O+	01434103697
	16	Anthony	Ram	M	AB+	04051531952

Fig. 7.1: Display all active patients

SELECT doctor_id, first_name, last_name, specialization, experience_years
FROM Doctors
WHERE experience_years>5;

	doctor_id	first_name	last_name	specialization	experience_years
▶	2	Liam	Dugal	Epileptology	15
	3	Liam	Setty	Sports Medicine	24
	5	Sai	Sidhu	General Dermatology	25
	6	Ojasvi	Suresh	Head & Neck Surgery	20
	7	Dipta	Kota	Internal Medicine	29
	8	Vritti	Ray	High-risk Obstetrics	13
	9	Chandran	Bedi	Medical Oncology	19
	10	Azaan	Murthy	Child Psychiatry	28
	11	Ekaja	Mital	Intervt Interventional Cardiology	
	12	Ekani	Lalla	Stroke medicine	10
	13	Dayita	Borra	Joint Replacement	13
	14	Ojas	Sabharwal	General Pediatrics	6
	15	Devika	Lal	General Dermatology	18
	17	Yasti	Nayak	General Physician	26
	18	Vincent	Bedi	General Gynecology	13

Fig. 7.2: Display doctors with more than 5 years of experience

```
SELECT
  d.doctor_id,
  CONCAT(d.first_name, ' ', d.last_name) AS doctor_name,
  d.specialization,
  dept.department_name
FROM Doctors d
JOIN Departments dept
ON d.department_id = dept.department_id;
```

	doctor_id	doctor_name	specialization	department_name
	1	Dayita Bakshi	Interventional Cardiology	Cardiology
	11	Ekaja Mital	Interventional Cardiology	Cardiology
	21	Sachi Mall	General Cardiology	Cardiology
	2	Liam Dugal	Epileptology	Neurology
	12	Ekani Lalla	Stroke Medicine	Neurology
	22	Dakshesh Gade	Epileptology	Neurology
	3	Liam Setty	Sports Medicine	Orthopedics
	13	Dayita Borra	Joint Replacement	Orthopedics
	23	Tarak Pathak	Spine Surgery	Orthopedics
	4	Nisha Kala	Neonatology	Pediatrics
	14	Ojas Sabharwal	General Pediatrics	Pediatrics
	24	Hitesh Pant	Neonatology	Pediatrics
	5	Sai Sidhu	General Dermatology	Dermatology
	15	Devika Lal	General Dermatology	Dermatology

Fig. 7.3: Display doctors along with their department names

```
SELECT
  a.appointment_id,
  CONCAT(p.first_name, ' ', p.last_name) AS patient_name,
  CONCAT(d.first_name, ' ', d.last_name) AS doctor_name,
  a.appointment_date,
  a.status
FROM Appointments a
JOIN Patients p
ON a.patient_id = p.patient_id
JOIN Doctors d
ON a.doctor_id = d.doctor_id;
```

appointment_id	patient_name	doctor_name	appointment_date	status
33	Chandran Sandal	Dayita Bakshi	2026-06-10	Completed
47	Rajata Bora	Dayita Bakshi	2025-07-28	Completed
104	Abdul Lal	Dayita Bakshi	2026-05-02	Cancelled
134	Sara Chad	Dayita Bakshi	2026-07-05	Scheduled
155	Naveen Kothari	Dayita Bakshi	2026-03-02	Completed
175	Ganga Mandal	Dayita Bakshi	2025-07-22	No-show
176	Laban Devan	Dayita Bakshi	2025-05-12	Cancelled
177	Vasudha Thaman	Dayita Bakshi	2026-05-24	Completed
23	Sai Sarna	Liam Dugal	2026-04-14	Completed
36	Vedant Saran	Liam Dugal	2026-02-01	Scheduled
109	Gaurav Chhabra	Liam Dugal	2026-02-03	Scheduled
121	Daksha Tripathi	Liam Dugal	2026-06-16	Completed
123	Balveer Kade	Liam Dugal	2025-12-18	Scheduled
130	Lucky Bala	Liam Dugal	2026-02-02	Completed

Fig. 7.4: Display appointments with patient and doctor names

CHAPTER 8: TESTING AND DATA VALIDATION

Record the test cases used to verify the database. Include constraint tests, referential integrity tests and query-correctness tests.

Table 8.1: Test Cases and Results

TC No.	Test Case Description	Input / Action	Expected Result	Actual Result	Status
TC01	PRIMARY KEY constraint – duplicate key value	INSERT INTO Departments (department_id, department_name) VALUES (1, 'Test Dept');	Row rejected; primary key value must be unique	ERROR 1062 (23000): Duplicate entry '1' for key 'Departments.PRIMARY' – row rejected	Pass
TC02	UNIQUE constraint – duplicate doctor e-mail	INSERT INTO Doctors (first_name, last_name, email) VALUES ('Test', 'Doctor', 'dr.dayita.bakshi0@cityhospital.com');	Row rejected; e-mail must be unique	ERROR 1062 (23000): Duplicate entry 'dr.dayita.bakshi0@cityhospital.com' for key 'Doctors.email' – row rejected	Pass
TC03	NOT NULL constraint – mandatory first name	INSERT INTO Patients (first_name, last_name) VALUES (NULL, 'Test');	Row rejected; first_name cannot be NULL.	ERROR 1048 (23000): Column 'first_name' cannot be null – row rejected	Pass
TC04	CHECK constraint – invalid gender value	INSERT INTO Patients (first_name, last_name, gender) VALUES ('Test', 'User', 'x');	Row rejected; gender must be M, F or Other	ERROR 3819 (HY000): Check constraint 'Patients_chk_1' is violated – row rejected	Pass
TC05	CHECK constraint – invalid appointment status	INSERT INTO Appointments (patient_id, doctor_id, department_id, status) VALUES (1, 1, 1, 'Postponed');	Row rejected; status must be Scheduled, Completed, Cancelled or No-show	ERROR 3819 (HY000): Check constraint 'Appointments_chk_2' is violated – row rejected	Pass
TC06	DEFAULT constraint – status of a newly added patient	INSERT INTO Patients (first_name, last_name) VALUES ('Test', 'User'); SELECT status FROM Patients WHERE first_name = 'Test' AND last_name = 'User';	status is filled in automatically as 'Active'	1 row returned: status = Active	Pass
TC07	Referential integrity – doctor with a non-existent department	INSERT INTO Doctors (first_name, last_name, department_id) VALUES ('Test', 'Doctor', 99);	Row rejected; department 99 does not exist	ERROR 1452 (23000): Cannot add or update a child row: foreign key constraint 'Doctors_ibfk_1' fails – row rejected	Pass
TC08	Referential integrity – appointment for a non-existent patient	INSERT INTO Appointments (patient_id, doctor_id, department_id) VALUES (999, 1, 1);	Row rejected; patient 999 does not exist	ERROR 1452 (23000): Cannot add or update a child row: foreign key constraint 'Appointments_ibfk_1' fails – row rejected	Pass
TC09	Referential integrity – deleting a parent row that has child rows	DELETE FROM Medicines WHERE medicine_id = 1;	Delete blocked; medicine 1 is used in PrescriptionDetails	ERROR 1451 (23000): Cannot delete or update a parent row: foreign key constraint 'PrescriptionDetails_ibfk_2' fails – delete blocked	Pass
TC10	Referential integrity – valid child row is accepted	INSERT INTO Appointments (patient_id, doctor_id, department_id) VALUES (1, 1, 1);	Row inserted; patient 1, doctor 1 and department 1 exist	Query OK, 1 row affected	Pass
TC11	Orphan-record check on the loaded data (Admissions – Rooms)	SELECT COUNT(*) FROM Admissions a LEFT JOIN Rooms r ON a.room_id = r.room_id WHERE r.room_id IS NULL;	0 – every admission refers to an existing room	0 – no orphan rows	Pass
TC12	Join correctness – every doctor appears exactly once	SELECT COUNT(*) FROM Doctors d JOIN Departments dp ON d.department_id = dp.department_id;	30 (equal to SELECT COUNT(*) FROM Doctors)	30	Pass
TC13	Aggregate correctness – GROUP BY totals add up to the table total	SELECT SUM(cnt) FROM (SELECT COUNT(*) AS cnt FROM Patients GROUP BY state) t;	100 (equal to SELECT COUNT(*) FROM Patients)	100	Pass
TC14	Billing correctness – invoice total = charges – discount + tax	SELECT COUNT(*) FROM Invoices WHERE ROUND(consultation_charges + room_charges + lab_test_charges + medicine_charges + other_charges - discount + tax_amount, 2) <> total_amount;	0 mismatching invoices	0 mismatching invoices (all 130 invoices tally)	Pass
TC15	Transaction correctness – ROLLBACK restores the old value	START TRANSACTION; UPDATE Rooms SET daily_charge = 0 WHERE room_id = 2; ROLLBACK; SELECT daily_charge FROM Rooms WHERE room_id = 2;	daily_charge is still 8000.00	daily_charge = 8000.00 (unchanged)	Pass

CHAPTER 9: GITHUB REPOSITORY AND TEAM CONTRIBUTION

9.1 Repository Details

GitHub Repository Link:

<https://github.com/Narasimhasai10/hospital-management-system.dbms>

9.2 Team Responsibilities and Contribution

Table 9.1: Team Responsibilities

S.No	Student Name	Roll Number	Role / Responsibility	Modules Contributed
1.	KOPPALA DEVI SRI CHARAN	25B11AI570	ER Diagram&Relational Schema	diagrams/er-diagram.png
2.	DANDE LENANDAN NARASIMHASAI	25B11AI247	DDL, Tables, Keys&Constraints	sql/ddl.sql
3.	BODDU SRIRAM RATNA KUMAR	25B11AI138	DML, CRUD, JOIN&Aggregate Queries	sql/dml.sql, sql/queries.sql
4.	KETHAMALLA VEERA CHANIKYA	25B11AI528	Documentation&Screenshots	docs/, screenshots/

9.3 Review-wise Progress

Table 9.2: Review-wise Progress Record

Review	Date	Deliverables Presented	Remarks of the Guide

CHAPTER 10: CONCLUSION AND FUTURE ENHANCEMENTS

10.1 Conclusion

The Hospital Management System database was successfully designed and implemented to manage important hospital activities in a centralized and structured manner. The database includes 14 entities/tables covering patients, doctors, departments, appointments, admissions, rooms, prescriptions, medicines, laboratory tests, invoices, inquiries and administrators.

The database design uses Primary Keys, Foreign Keys, UNIQUE constraints and CHECK constraints to maintain data accuracy and referential integrity. The database is normalized to reduce data duplication and improve consistency between related tables.

Various SQL operations were implemented, including DDL operations for creating tables and constraints, DML operations for inserting, updating, deleting and retrieving data, and SQL queries using JOINS, aggregate functions and subqueries for obtaining useful information from the database.

The objectives stated in Section 1.4 were achieved by creating a centralized database that manages hospital records efficiently, maintains relationships between different entities, reduces data redundancy, supports data validation, and allows information to be retrieved using SQL queries. Thus, the project provides a structured database foundation for managing hospital operations and can be further extended into a complete hospital management application.

10.2 Limitations of the Present Work

The current work mainly focuses on the database design and SQL implementation of the Hospital Management System. It does not currently include a complete web or mobile application interface for hospital staff and patients. Real-time notifications such as appointment reminders and alerts are not implemented. Online payment

processing and integration with external hospital systems are also not included in the current version. In addition, automated database backup, recovery, advanced analytics and reporting dashboards are not implemented. These features can be added in the future to make the system more complete and suitable for real-world hospital use.

10.3 Future Enhancements

The Hospital Management System can be enhanced in the future by developing a complete front-end application connected to the existing database. Additional modules can be added for managing pharmacy, billing, laboratory reports and patient services. Stored procedures and triggers can be implemented to automate frequently performed database operations and maintain data consistency. Proper indexing can also be added to improve query performance as the amount of data increases. Security can be improved through user authentication, role-based access control and better protection of sensitive patient information. Automated database backup and recovery mechanisms can be introduced to prevent data loss. Reporting and analytics features can also be added to generate useful reports and help hospital staff analyze patient, appointment, billing and other hospital data.

REFERENCES

List textbooks, reference books, documentation and websites referred to. Follow the format shown below.

6. Abraham Silberschatz, Henry F. Korth and S. Sudarshan, “Database System Concepts” , 8th Edition, McGraw-Hill, ISBN: 978-1260230508.
7. Raghurama Krishnan and Johannes Gehrke, “Database Management Systems” , 4th Edition, McGraw-Hill, ISBN: 978-1260091195.
8. Ramez Elmasri and Shamkant B. Navathe, “Fundamentals of Database Systems” , 7th Edition, Pearson, ISBN: 978-0137504277.
9. MySQL Documentation, “MySQL 8.0 Reference Manual” , Oracle Corporation.
[MySQL 8.0 Reference Manual](#)
10. W3Schools, “MySQL Tutorial” , W3Schools. [MySQL Tutorial](#)

APPENDIX A - COMPLETE SQL SCRIPT

Paste the complete, final SQL script (CREATE, INSERT and all queries) in order, or give the exact path to the script file in the GitHub repository.

<https://github.com/Narasimhasai10/hospital-management-system.dbms/blob/main/sql/ddl.sql>